The Norns III: Packing with the HDL's Structure, a Known-Optimum Yardstick, and the Leak It Found

Peter Jamieson
Electrical and Computer Engineering, Miami University
Oxford, OH, USA
jamiespa@miamioh.edu

Abstract—Before an FPGA design is placed on a chip, the grouping the designer wrote down in the source is mostly lost. This paper tests, one hypothesis and one experiment at a time, whether keeping it helps. It contributes (1) a packer, a copy of VPR's own, with a bonus for keeping the pieces of one HDL register together; (2) a yardstick of designs built so that the best grouping is known, which shows that the standard packer uses 22% more wire than the known grouping over all topologies and 49% more on meshes and trees; and (3) a finding about method. The packer's first measured gain (wirelength 0.902 of stock) was a leak: the input file already equalled the answer. Re-run with registers that cut across the blocks, the packer used 1.65 to 2.51 times the wire of the standard packer at every register width we tried, and with perfectly aligned registers it used 0.669 of it. Near-critical connections are kept inside tiles more often than connections in general, on 9 designs. A sweep of VPR options adds eight area and speed effects. Starting placement from the designer's grouping, the idea this work began with, has no measured gain. The packer's own idea of which connections are critical is mostly stale: 81% of its picks on critical connections hold nothing that is critical after routing, and refreshing the criticality while packing did not lower the delay. The yardstick designs are too regular to show the true gap (their Rent exponent is 0.08, against 0.56 for real designs), so a second yardstick is built from real designs. Speed is judged by the critical-path delay and by near-critical paths; power is not measured. Of 64 hypotheses, 14 are useful, 9 possibly good, 23 inconclusive and 18 dismissed; nine further rows are measurements and ten ideas are untested.

Index Terms—FPGA packing, placement, provenance, known-optimum benchmarks, VTR

I. Introduction

An FPGA is a chip made of many small programmable logic blocks joined by programmable wires; a design is loaded onto it by setting those parts. A designer writes the design in a hardware description language (HDL), and a chain of CAD programs turns that text into a pattern on the chip. Two steps of the chain decide most of the result. Packing groups small pieces of logic into tile-sized groups, and placement decides where on the chip each group sits. Packing and placement see the design as a long list of gates and wires. The grouping the designer wrote down (this register, that adder, that block of the source) is mostly gone by then.

This paper asks one question: does keeping the designer's grouping make packing and placement better, and how can we find out honestly? We answer it with hypotheses. Each hypothesis is stated in plain language, an experiment is run, and the experiment either confirms it or does not. Ideas that the experiments confirm are in the main text; ideas that failed are one line in the research log (Appendix A).

What good means. A real design is judged on three things at once: the chip area it needs (low), its speed (high) and its power (low). We measure area and speed. Power is not measured in any experiment of this paper. Speed means the slowest path of the design and also the paths nearly as slow, because speeding up the slowest path alone only promotes the next one (Section IV).

The data behind it. Mímir III is the dataset this work feeds. Every piece of the circuit carries a record of the HDL text it came from. Mímir stores that record beside where each piece was placed and how fast its paths are, so the data can show why things ended up where they did, and not only where. Section IV-F says what Mímir holds today and what it lacks.

The innovations, in plain English.

1) A packer that knows the grouping. It is a copy of the standard open-source packer with a bonus for putting pieces of one HDL register or bus into the same tile (Section III-A).
2) A yardstick with a known best grouping. We build designs out of blocks that each exactly fill one tile, so the best tile grouping is known, and measure how far a packer is from it (Section III-B).
3) A leak found by the yardstick. The first test of the new packer showed a gain of nearly ten percent in wire used. The yardstick showed that the gain came from an input file that already contained the answer: a leak. With groupings that do not mirror the answer, the packer used more wire at every setting we tried (Section III-E). We state this plainly because it is a lesson about method: a gain that exists only when the tool is handed the solution is not a gain.
4) A count of what the standard packer keeps together on slow paths (Section III-F).

5) A negative baseline. Starting placement from the designer's grouping instead of the tool's own start did not help (Section III-G).
6) A second yardstick from real designs. The synthetic yardstick is too regular: its wiring hardly changes with the size of the region around a block, as real wiring does (Section III-D). So we edit real designs until their grouping is the known best (Section III-C).
7) Where the greedy packer goes wrong. We measure the connections the packer treats as nearly the slowest, show that most of its picks from them are for connections that are not slow in the finished design, and keep a catalogue of the HDL patterns for which the pick goes wrong (Sections V-G and V-H).

Contributions. Each is an innovation with the hypotheses that the experiments confirmed. (1) The standard packer scatters the source's registers across tiles (H8), and the new packer keeps them together without using more tiles (H9a; mechanism checks, not end results), all useful. (2) The yardstick is exact (H24, H27, H29–H31: useful) and shows room: the standard packer uses 22% more wire than the known grouping over all topologies and 49% more on meshes and trees (H25/H19, H32: useful); the new packer still leaves room (H20: useful). (3) The new packer's gain exists only when the HDL registers equal the blocks (H33, H36, H35: useful, an upper bound and a harm finding), and the re-run with realistic registers did not confirm a gain (H22′: dismissed). (4) Connections nearly as slow as the slowest cross tile boundaries less often than connections in general under the standard packer (H17c: possibly good, fragile). (5) A sweep of tool options gives eight possibly good area and speed effects (Section V-J). (6) The real-design yardstick is built on 33 real designs and shows the standard packer needs about 10% more CLBs than the known clustering and about 27% more wire (H38: useful). (7) The packer's idea of which connections are the slowest is fixed before packing and is stale for 81% of its picks from them (HP-1, a measurement), and refreshing it while packing does not lower the delay (H-refresh: inconclusive when refreshed after every group packed, dismissed after every eighth). In total the research log has 73 rows: 14 useful, 9 possibly good, 23 inconclusive, 18 dismissed, nine measurements, and ten untested ideas listed in Section VII.

II. Background

This section defines every term of the paper once, in dependency order, so that no term is used before its definition. The basic definitions follow the order and the wording of the FPGA and CAD background chapters of two University of Toronto theses of the same research line [1], [2]: the soft logic fabric and the basic logic element, the three CAD stages that follow synthesis, and (from Luu) the timing terms. The terms that are the paper's own come after them.

A. Glossary

FPGA A chip made of many small programmable logic blocks joined by programmable wires; a design is loaded onto it [1].

netlist A list of the logic gates of a design and the wires between them [1].

HDL Hardware description language: the Verilog text a designer writes.

LUT Look-up table: the small memory that implements any logic function of a few inputs [1], [2].

K The number of inputs of a LUT; here $K = 6$.

BLE Basic logic element: one LUT with its flip-flop [1], [2].

cluster A set of netlist blocks that are meant to belong together; a clustering is a partition of the netlist into clusters.

cut The number of nets whose pins lie in more than one cluster of a clustering; fewer is better.

Rent exponent The p of Rent's rule $T = k\,B^p$, fitted to the terminals T (nets leaving a block) of blocks of B netlist nodes found by recursive bisection; real logic has p near 0.5 to 0.7, and p near 0 means a block's size hardly changes how many nets leave it.

CLB Configurable logic block: a group of 10 BLEs placed as one unit (a logic block or soft logic cluster tile in the theses).

module One HDL module, the unit of the design hierarchy.

provenance The record kept on every netlist object of which HDL text, and which module, it came from.

word A group of netlist blocks that are the bits of one HDL vector (a register, an adder or a bus).

io-word A word made from a top-level input or output bus, extended inward to the logic next to it.

placement The choice of which site on the chip each block occupies; a placer makes it [1].

routing The choice of which wire segments connect the placed blocks; the router makes it [1].

VPR The place-and-route program of the VTR flow [3]–[5].

wirelength The total length of the routed wires of a design (WL); lower means less routing area and usually less delay.

critical path The slowest register-to-register path of a design; it sets the clock speed.

CPD Critical-path delay: the delay of the critical path, in nanoseconds.

slack How much earlier than required a signal arrives; negative slack means the clock is too fast. VPR normalises slack by the CPD [2].

criticality A connection's closeness to the CPD, 1 − slack/CPD, so 1 is on the critical path; the criticality of an atom is the largest criticality over the paths (post-route, from the 1000 reported paths) or input pins (pack time) that contain it [2].

near-critical A path (or connection) whose delay is within a stated fraction of the CPD; criticality is 1 − slack/CPD [2], and VPR 9 relaxes it per timing constraint [6].

channel width The number of routing tracks per channel; a smaller routable width needs less wire area.

packing The step that groups LUTs and flip-flops into CLBs before placement; VPR's packer is AAPack [1], [2].

simulated annealing A placement search that accepts worse moves at a falling temperature, to escape bad layouts.

analytic placement A placement computed by solving a smooth wirelength model; VPR's starting point for the anneal [5].

affinity packing The Norns packer: a copy of VPR's packer with a bonus β for adding atoms of the same word or module to a CLB (the affinity file names the group of each atom).

purity The share of a CLB's labelled atoms that carry its most common word (or module), atom-weighted over CLBs.

split factor How many CLBs a word spans, divided by the fewest CLBs that could hold it; 1.0 is best.

known-optimum yardstick A design built from blocks that each fill exactly one CLB, so the intended clustering is known by construction; it is an optimum for CLB count only and not a proven best placement.

foreign clustering A clustering that VPR did not make (here METIS), used to build a yardstick so that VPR is not graded on its own answer.

alignment How closely the HDL words match the blocks of a yardstick design: 1 means each word is one block, 0 means each word holds about ten blocks' bits.

leak A gain that exists only because the input file (the word file) already equals the answer (the block partition).

pairing class The class of a pair of atoms in one CLB: series-critical, series-noncritical, shared-input, orthogonal or filler.

critical heap The set of atoms whose criticality is at least 0.90 (the near-critical atoms the greedy packer takes first). The picture: the packer works the heap, a new heap forms as it goes, and at the end a heap remains whose criticality has been pushed down to a floor it cannot get below (the cell delays alone).

pack-time heap The critical heap as the packer sees it: atoms whose pack-time criticality pc is at least 0.90, from VPR's own timing analysis before packing, in which every net between logic blocks has one delay.

post-route heap The critical heap of the routed design: atoms whose criticality rc is at least 0.90 in the 1000-path timing report.

stale pick A molecule the packer adds to a cluster on a pack-time criticality of at least 0.90 that holds no atom of the post-route heap; the packer's view of what was critical was out of date or wrong.

exception HDL pattern A detectable HDL construct, found before packing, for which grouping by HDL structure is a bad packing hint or for which the greedy pick is known to go wrong (for example a parallel bus word whose bits share no dataflow).

seed A placement file given to VPR to start the anneal from, instead of its own start; soft seeds may be left by the anneal and hard seeds are floorplan regions it must keep.

Urd The provenance seed: its clusters are the HDL modules.

Verdandi The recovered seed: clusters found from the flat netlist alone (Loki III L2: Louvain, spectral, affinity propagation, k-means) [7].

quotient layout A die layout of clusters: contract each cluster to a node, weight edges by cut nets, lay the small graph out and scale it to the die.

scale S The anneal's start temperature as a multiple of VPR's own automatic estimate.

crossover The largest tested scale S at which a seeded cold start still routes shorter than an unseeded run.

retention The fraction of a seed's CLBs still inside the seed cluster's region when the anneal ends.

hindsight clustering The stock clustering with atoms moved or swapped between neighbouring CLBs according to where the best stock placement put them.

flat-placement file VPR's one-line-per-atom placement file (–read_flat_place), through which an outside placer enters VPR.

nextpnr An open-source FPGA placer with an analytic mode (HeAP) and an annealing mode (SA) [8], [9].

reread control The control that reads VPR's own final placement back into the default flow.

rule H0 Adders, multipliers, RAMs and pads are fixed at VPR's analytic positions for the external placers.

run record A mimir-run.json written for each VPR or nextpnr run: commands, tool commits, parameters with their source, hashes.

geomean ratio The geometric mean over designs of variant over reference; below 1 favours the variant for a lower-is-better metric.

sign test An exact two-sided test on how many designs improve; ties are left out.

Wilcoxon test A rank test of the paired per-design differences (one-sided where the text says so).

Holm adjustment The correction of p-values for several simultaneous tests.

bootstrap interval A confidence interval from resampling the designs.

verdict rule The numeric rule that maps a result to supported, not supported or inconclusive.

Three more terms are used as plain words and need no entry: area is the LUT count, the CLB count, the die grid

and the channel width; speed is the CPD and the near-critical statistics; power is not measured (Section IV-C).

B. Concepts

The flow runs in this order. The front end reads the HDL and writes a netlist (here Odin III, with the provenance record on every object). Packing groups the netlist into CLBs. Placement puts the CLBs and the fixed blocks on the die: VPR starts from an analytic placement and refines it by simulated annealing. Routing connects the placed blocks, and timing analysis reports slack and the CPD [3]–[5]. The designer's grouping is a property of the HDL. It can be lost at each step: the front end may drop the register boundaries, packing mixes registers in a CLB for connectivity, and the anneal moves CLBs freely.

Two earlier lines of work connect to this question. Gharibian et al. recovered system-level structure from flat netlists and showed that it correlates with placement locality [10]–[12]. Cong, Romesis and Xie built placement problems with known optima to measure how far placers are from them [13]; our yardstick follows that idea for packing. RLPlace improves the anneal itself with directed moves and checkpointing [14]; we leave the anneal as it is.

C. Where the Norns sits in the suite

Figure 1 places The Norns III in the suite. Yggdrasil III is drawn as a tree. Odin III is the trunk, and the provenance record that the packer uses comes from its one IR. The Norns III is a side branch, a tool and experiment for placement with provenance, next to Loki III (reverse engineering, whose L2 clusterings are the recovered seeds), Leman Russ III and Verdandi III. It runs Odin III and VPR as programs and changes neither. Mímir III, the circuit knowledge base, hangs from the canopy as the fruit, and the Norns' run records are among what it stores. Urdarbrunnr, the well of FPGA CAD and its views (VTR and VPR, nextpnr, DREAMPlaceFPGA, OpenPARF), feeds The Norns III.

III. The Innovations

Each subsection gives the idea in plain English first and the technical description second. Which experiments support each innovation is in Section V.

A. A packer that keeps the HDL's registers together

Plain English. Before placement, a tool packs small pieces of logic into tile-sized groups. The standard packer chooses pieces that share wires. It does not know that eight flip-flops are the eight bits of one register, so it can scatter them over many tiles even though they will be read and written together. Our packer is a copy of the standard one that adds a bonus for putting pieces of the same register, bus or module into the same tile.

Technical. The Norns packer is a patch to VPR's AAPack clusterer (VTR commit 3c9a4d23b2, 8 changed files, 536 added lines; patches/). Each atom carries a group id read from the affinity file (–norns_affinity); the group is the module, the word, or the word with io-words (source module, word, word+io). A CLB's dominant group is the plurality group of its atoms so far. A candidate molecule gains $+\beta$ (–norns_affinity_weight) for every atom in that group, and up to 30 unclustered members of the group are proposed as candidates, because the bits of a word are parallel and no net links them. The stock gain lies in $[0, 1]$, so β is in the units of one connected atom. Legality stays with AAPack's legaliser. With the option absent or $\beta = 0$ the packer must write a .net byte-identical to stock; this was checked on 60 comparisons over 15 designs before any experiment. Words come from Odin III's provenance record; io-words extend a top-level bus inward over LUT and flip-flop atoms (two hops past the atom next to the pad, through nets of fanout at most 16).

B. A yardstick with a known best grouping

Plain English. To know whether a packer is good we need designs where the best grouping is known, so that the packer can be scored against it. We have two such yardsticks. The first (this subsection) is synthetic: we build designs out of blocks that each exactly fill one tile, wired as chains, meshes, trees or random graphs. Because every block fills one tile, no packer can use fewer tiles than there are blocks, and the block grouping reaches that count; the tile count of the best grouping is therefore known exactly. That is the whole claim. It is not a proven best placement, not a proven best wire length and not a proven best speed: the known grouping is handed to the same placer and router as every other grouping, and what comes out is what that tool achieves from it. A packer may even beat it on wire or speed. The same logic can be written in the HDL with its registers lined up with the blocks, or deliberately misaligned, and that turns out to matter a great deal (Section III-E). The second yardstick, built from real designs, follows in Section III-C.

Why the synthetic count is exact. A block is ten six-input functions, each feeding one flip-flop. A six-input function needs a whole logic element of the architecture (the two-LUT mode shares five input pins and takes five-input functions only), and the flip-flop can only be fed by its own function, so the ten functions need ten logic elements, one CLB. A design of B blocks therefore needs at least B CLBs in every packing, whatever the packer, and the block partition uses exactly B. This is a counting argument and does not depend on the packer. It does not cover wire length or delay, and it does not prove that the designed cut (the nets that leave a block, a few per block) is the smallest possible.

Technical. A block is 10 six-input functions, each feeding a flip-flop (ten logic elements of the k6_frac_N10 architecture, so exactly one CLB), with dense internal connections and at most two external inputs per function; the CLB count B is an exact lower bound reached

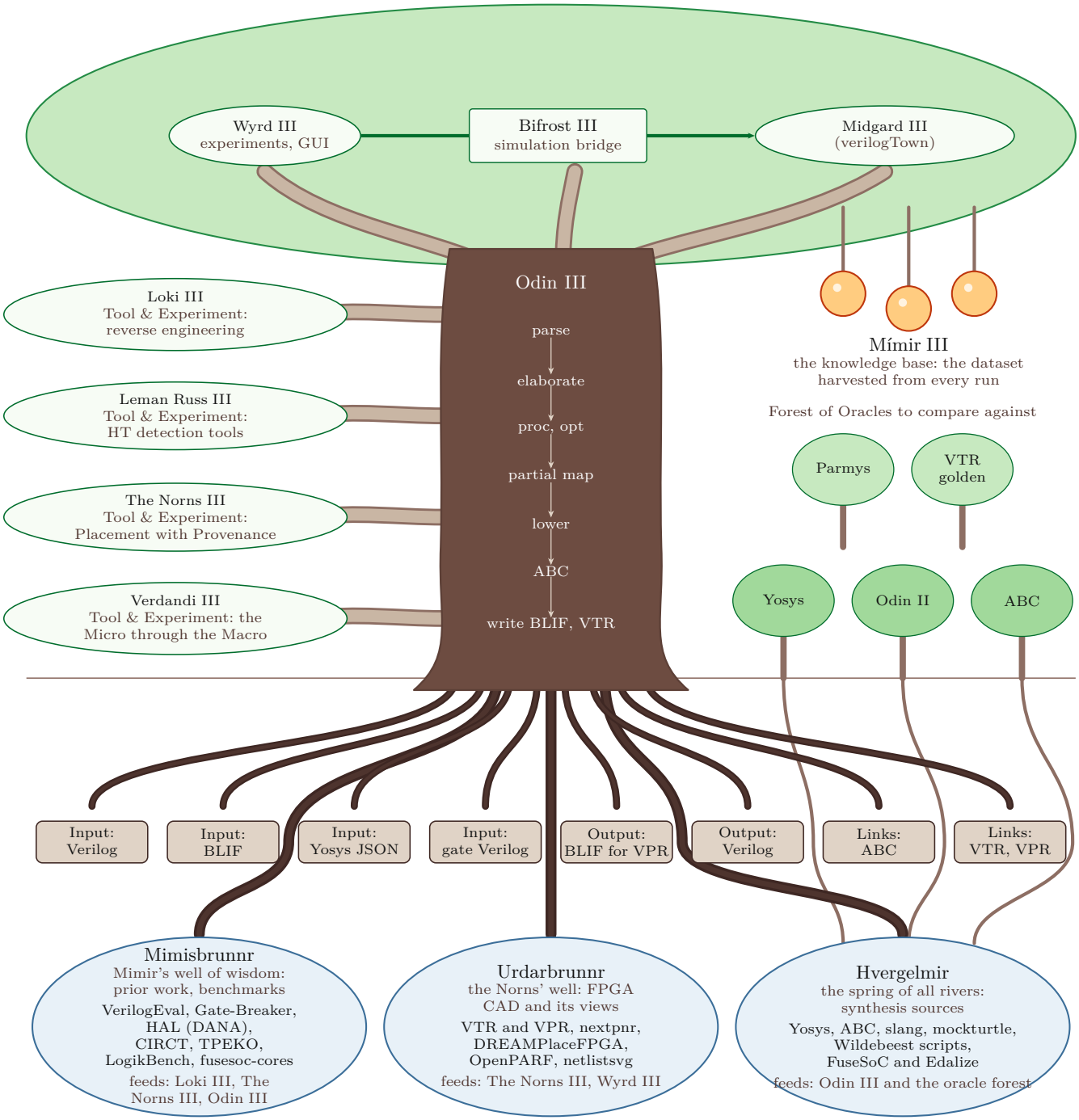


Fig. 1. Yggdrasil III drawn as a tree, and where The Norns III sits in it. The trunk is Odin III (https://github.com/drpaj12/odin3); the roots are its inputs and the tools it links to, and three wells hold outside work. The oracles stand beside the trunk. The canopy holds Wyrd III (https://github.com/drpaj12/wyrd), Bifrost III and Midgard III, and the fruit, Mímir III (https://github.com/drpaj12/mimirIII). Side branches carry tools and experiments: Loki III (https://github.com/drpaj12/loki), Leman Russ III, The Norns III (https://github.com/drpaj12/nornsIII, this paper) and Verdandi III (https://github.com/drpaj12/verdandiIII). The hub is https://github.com/drpaj12/YggdrasilIII.

by the block partition. A generator (gen/r5gen.py, seed 20261026) writes 72 designs: four topologies (chain, mesh, tree, random sparse), nine sizes $B = 4$ to 1024, two styles (dp: a datapath slice whose HDL has the vector r[9:0]; ctl: scalar registers), at two levels (NL: the netlist written directly as BLIF; HDL: the same specification as Verilog through Odin III). At the netlist level the check also requires that swapping any two logic elements of different blocks never lowers the cut. The known arm hands the block partition to the Norns packer as affinity groups with $\beta = 100$; the ratios in this paper are against the placement VPR produces from that clustering, so they are a yardstick and not a bound. A second generator (gen/r6gen.py) fixes the logic and varies only the HDL declarations: at alignment a a fraction $1 - a$ of each block's flip-flops is displaced into other words, at $a \in \{0, 0.25, 0.5, 0.75, 1\}$ for mesh, tree and chain with seven sizes (105 designs). The idea follows Cong, Romesis and Xie [13].

What it found, and what it cannot claim. The standard packer uses 1.073 times the CLBs, 1.220 times the wire and 1.093 times the delay of the known grouping over the 72 designs, and 1.489 times the wire on meshes and trees (Section V-B). The synthetic designs are regular by construction: every block is the same size, the wiring between blocks is a fixed pattern, and, as Section III-D measures, they have almost no Rent growth. A regular design may hide the true gap, in either direction, and the 1.489 for meshes and trees against 1.0 for chains and random graphs shows that the gap depends on the pattern. This is why the second yardstick starts from real designs.

C. A yardstick built from real designs

Plain English. Take a real design and a grouping of its logic into tiles (what VPR's own packer made, or what an independent partitioner made). Then edit the design, touching only the inside of each group, until that grouping is provably the best one for tile count and a strict local optimum for wiring between groups. The wiring between groups, which is the real design's own structure, stays as it was. We then hand the edited design to the standard packer and to our packer told the grouping, and compare. Three edits do the work. (1) Pad every group with dummy logic until it is exactly full, so no packer can use fewer tiles than there are groups. (2) Make every look-up table use all six inputs, so two small tables cannot share one logic element. (3) Add connections inside each group, never between groups, so that moving any single logic cell out of its group makes the number of cut nets rise. The idea is that of Cong, Romesis and Xie [13]; we apply it to real structure because the synthetic yardstick is too regular (Section III-D).

Technical. The design D is the Odin III netlist after VPR's own clean-up. A clustering C_0 is either own (the clusters of VPR's standard packer, seed 1) or foreign (METIS k-way on the cell hypergraph, the lowest cut of 40 seeds followed by pair-swap descent, with N_C the number of own clusters (called K in the pilot data files)). It is repaired for six-input accounting (a cluster of fourteen fractured small LUTs now needs fourteen logic elements) into C with N_C clusters, so N_C can exceed VPR's own cluster count; clusters respect at most 32 external input nets and 12 external output nets, which the default flow can form, and carry chains are cut into segments of at most 12 adders (one carry net removed per boundary, counted). Each cluster is padded with dummy cells (a six-input LUT feeding a flip-flop) to exactly 10 logic elements, with dummy inputs only from nets driven inside the cluster. The CLB bound is then exact by counting: a six-input cell cannot share a logic element, a flip-flop is fed only by its own LUT, hard blocks are other block types, and a CLB has 10 logic elements, so every legal packing needs at least $10N_C/10 = N_C$ CLBs and C reaches N_C. The inside is strengthened by filling every LUT to six inputs from nets driven inside its cluster, doubling a net that has a single pin in a cluster, and anchoring each cell to a dummy's output. No net that leaves a cluster is added or rewired, except that one pad or hard-block input of a full cell may be replaced by a dummy output (counted).

Margin, check and shuffles. Among the partitions with the minimum number of CLBs every part holds exactly 10 logic elements, and for those the construction proves a swap margin: swapping any two cells of different clusters raises the cut by at least the margin computed from the shared nets (the measured minimum is 1 or 2 on every pilot build), so C is a strict local optimum of the single-swap neighbourhood when no cell is left unanchored. A cell with no spare input slot and no dummy to read cannot be strengthened without rewiring real nets; such cells are counted as unmet and flag the design (0 to 3 cells in four of the six pilot builds, 14 in the foreign build of diffeq1). The optimum is not proved global. As a check, 100 METIS runs (repaired to exactly full parts and refined by pair-swap search) and 100 restarts of a swap search from C with 10% of the cells perturbed were run per design; none found a smaller cut. For diffeq1 no METIS partition satisfied the pin limits, so that check is vacuous there and the search from C is the evidence. The result is evidence and not proof. To remove file order as a clue the BLIF atom and net names are replaced by random tokens and the statement order is permuted, three shuffles per design, plus an unshuffled ordered control. To expose self-bias, the foreign construction asks whether VPR does relatively better on a grouping that is its own answer than on one that is not.

What it can and cannot claim. Area is exact (CLB count, by counting). The cut is optimal by margin against single swaps and checked, not proved, against larger rearrangements. The critical-path delay is observed and not claimed: the known arm's delay is what VPR achieves from C, and ratios below 1 are possible. Hard blocks and

TABLE I
Rent exponent rent.p by design set (three partitioner seeds per design; each seed is one run). Negative runs have a fitted exponent below 0; low-confidence runs are flagged by the measurer (a fit on fewer than five points); the last column is the mean over the runs that are not flagged. Source: data/analysis/rent-split.json, from the Mimir main store.

Set	n	Mean p	Range	Neg.	Low	Conf.
Real VTR	12	0.562	0.44 to 0.70	0/36	9/36	0.562
MCNC	20	0.586	0.32 to 0.75	0/60	36/60	0.608
Loki LUT	10	0.617	0.44 to 0.76	0/30	21/30	0.468
Loki gate	22	0.606	0.37 to 0.72	0/66	48/66	0.583
Synth. r5	47	0.126	−0.55 to 0.56	39/141	45/141	0.203
Synth. n5	48	0.102	−0.54 to 0.56	48/144	48/144	0.184
Synth. r6	90	0.038	−0.52 to 0.48	105/270	90/270	0.101
Synth. all	185	0.077	−0.55 to 0.56	192/555	183/555	0.149

pads are outside the claim (each net between them and logic is cut in every grouping and is a constant). The pilot and its full-set slot are in Section V-C.

D. Why the synthetic yardstick is regular: Rent's rule

Plain English. Real circuits get more connected to the rest of the chip as a block grows, at a rate that Rent's rule describes by one number, the Rent exponent. A synthetic design made from identical blocks with a fixed handful of wires between them has hardly any such growth. If the packer's gap is measured only on such designs, we cannot tell how big the gap is on real logic. We measured it.

Technical. The exponent is the Mimir metric rent.p (recursive bisection with METIS, three partitioner seeds), read from the Mimir main store without modification and split by design set by scripts/rent_split.py (output data/analysis/rent-split.json). The synthetic designs have a fixed number of external inputs and outputs per block, and their nets stay inside blocks or follow the block pattern.

Result. Real designs have a Rent exponent of 0.32 to 0.76 whichever set they come from (mean 0.56 for the 12 VTR Verilog designs of this paper that are large enough to measure, 0.59 for MCNC, 0.61 to 0.62 for the Loki designs). The synthetic designs have a mean of 0.04 to 0.13, and 192 of their 555 runs (64 of 185 designs) give a negative exponent, which no real design in the table does. Restricting to the runs the measurer does not flag raises the synthetic mean only to 0.15 against 0.56 for the real designs, so the gap is not an artefact of the low-confidence fits (every one of the 144 n5 runs is flagged, which is itself a statement about how small their curves are). The reading is that the synthetic designs have almost no Rent growth, because each block has the same few external wires whatever the size of the region around it, so their regular structure can hide the true gap: a packer is not tested on the property of real logic that makes grouping hard, the way terminals grow with region size. This is the reason for building the second yardstick from real designs (Section III-C). It is a limit of the first yardstick and not a claim about its numbers: those stand as the gap on regular designs (Section V-B).

E. A leak found by the yardstick

Plain English. In our first test of the packer on the yardstick designs, the packer used about ten percent less wire than the standard packer. That looked like a result. It was not. In those designs the file that tells the packer which pieces belong together was written from the same blocks that define the best grouping, so the packer was handed the answer. Removing the answer and giving the packer registers that cut across the blocks, the packer used between 65% and 150% more wire. We report this as a finding about method: always check what the tool is given, and build the yardstick so that the hint can be dialled from perfect to useless.

Technical. The leak is word-file purity against the blocks near 1: 0.9998 in the dp designs, and the original gain over 63 designs came only from the 18 mesh and tree dp designs among them (docs/plan/round8-h22.md). The alignment ladder of the previous subsection varies the purity from 1 to 0.1. The H22′ re-run replaces the HDL register declarations by word buses of width $w \in \{4, 8, 16, 32\}$ over a random permutation of the flip-flops (no width is a multiple of 10, so words cut across blocks), for mesh and tree (test) and chain (control), seven sizes each: 84 designs, arms stock, norns ($\beta = 4$) and known, three seeds (gen/r8gen.py).

F. What the standard packer keeps together on slow paths

Plain English. A direct count of which kinds of pairs of logic pieces the standard packer puts in one tile: pairs on a slow path in series, pairs that share an input, and pairs that merely fill space. We also ask whether the connections that are almost as slow as the slowest are kept inside tiles more often than average connections.

Technical. A log-only census in the packer (–norns_pack_census) records, for every molecule added to a CLB, the candidate source and the gain terms. Every pair of LUT, flip-flop and adder atoms in one CLB falls in the first applicable pairing class: series-critical (a local net of fanout at most 32 from one to the other, pack-time sink criticality at least 0.5), series-noncritical, shared-input, orthogonal (same word, or sinks of one broadcast net), filler. Fifty random packings with the same CLB sizes give the chance level. Post-route criticality comes from Odin III's critical-path attribution of the routed runs. Near-critical connections are those within 10% of the CPD in the routed report.

G. Starting placement from the designer's grouping (a negative baseline)

Plain English. The simplest use of the designer's grouping is to give the placer a rough starting layout that keeps the

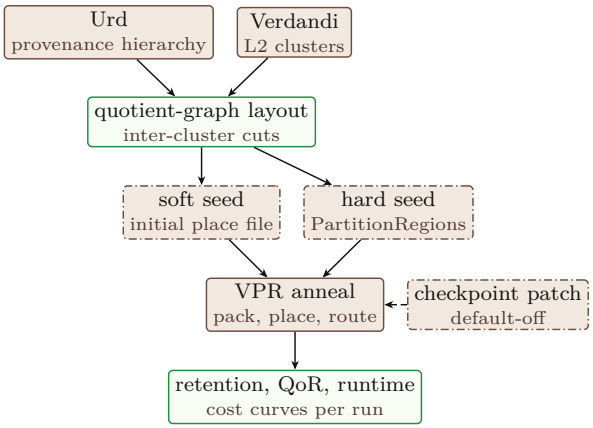


Fig. 2. The seeding baseline. Seeds from provenance and recovered clusters, arranged by a quotient-graph layout, enter VPR softly (initial placement) or hard (constraints). The crossover sweep cold-starts from the best placement and raises the initial temperature until the seed's advantage dies or the sweep reaches its cap.

groups together, and then let the heated search refine it. We ask how hot the search may start before it throws that layout away. This was the first idea of the Norns work. Its experiments are negative, and it is kept as the baseline that the packer is compared against in spirit.

Technical. A seed is a partition of the placed blocks: the provenance hierarchy (Urd) or a Loki III L2 clustering (Verdandi: Louvain, spectral, affinity propagation, k-means at k, $2k$, $4k$ with k the module count). The quotient layout puts clusters on the die; each cluster takes the free CLB slots nearest its laid-out point. A soft seed enters VPR as an initial placement file (–read_initial_place_file); a hard seed as one PartitionRegion per cluster (–read_vpr_constraints); pads and multiplier tiles stay anchored and carry chains move as one vertical run. The crossover sweep cold-starts the anneal from P^*, the best final placement over all seeds, raises the scale S and finds the crossover; all seeds are then run at $2S$ against an unseeded run at the same temperature (Figure 2).

H. Outside placers and run records

Plain English. To tell whether a finding belongs to VPR or to placement in general, we run a placer from outside the VTR tools, feed its result back into VPR, and let VPR route and time it, so every placer is judged by the same router and timer. Every run writes a record of what was run and where each parameter came from, so Mímir can store it.

Technical. A converter writes the outside placement (nextpnr HeAP or SA) in the flat-placement file format (–read_flat_place); a reread control reads VPR's own final placement back, and each ratio is within a family (analytic with analytic, annealing with annealing). Rule H0 fixes adders, multipliers, RAMs and pads at VPR's analytic positions, so the outside placers inherit part of VPR's solution and nextpnr has no timing model (both biases are in every table caption). Each run writes a mimir-run.json (scripts/runrecord.py).

IV. Methodology

A. Setup

Every experiment targets the VPR architecture k6_frac_N10_frac_chain_mem32K_40nm ($K = 6$, ten BLEs per CLB, carry chains, 32 Kb memories, multipliers). The netlist is the VTR-dialect BLIF that Odin III's front end writes: read_verilog –nolatches –vtr-compat, proc, fsm, opt, flatten, opt -full, wreduce, hardmap, lower, abc –lut 6, stats, write_blif –vtr –lut-k 6 –name-style provenance. The seeding baseline runs on VPR v8.0.0+18416 with one deterministic VPR seed per configuration. The packer experiments use VTR commit 3c9a4d23b2 (VTR [3], VTR 9 [15]) with three VPR seeds per configuration unless the experiment says otherwise. The designs are: the 15 VTR 8.0 designs of Table II; the generated yardstick designs (72, 105 and 84 designs, Section III-B); and for the option sweep 35 designs, the 20 MCNC circuits and the 15 designs of Table II. Every number in the paper traces to a committed file, named in a comment at the number in the source, in the research log (docs/paper/hypotheses.md) or in data/. Raw run directories are not committed; their summaries are.

B. Benchmarks

Table II reports each design's construction and its post-technology-mapping size. The module count is the front end's, while the CLB count and the grid are VPR's own numbers for the packed netlist.

The 15 designs span two orders of magnitude after technology mapping: from 13 CLBs (ch_intrinsics) to 1956 (stereovision2), 4843 CLBs in total. The mean is 322.9 CLBs and the median 125; the mean sits well above the median because three stereovision designs carry most of the logic. VPR auto-sizes a square grid to the packed netlist, with a mean side of 27.2 tiles (median 20) and a mean of 1076.4 grid tiles (median 400). Grid size follows the largest resource demand, not the CLB count: I/O-heavy and memory-heavy designs such as mkDelayWorker32B get a large die for few CLBs.

C. Metrics

Area (low is better). The LUT count of the Odin III netlist, the number of logic CLBs after packing, the die grid VPR chooses, and the minimum routable channel width.

Speed (high is better): critical and near-critical. The speed numbers are the critical-path delay (CPD) in nanoseconds and the near-critical statistics below. The routed wirelength (WL) is reported beside them as a proxy

TABLE II
Benchmark construction and post-technology-mapping size. All fifteen designs are from the VTR 8.0 benchmark set (vtr_flow/benchmarks/verilog). Modules is the front end's module count, CLBs the clb blocks VPR packs, Memory and Mult the memory and mult_36 blocks, and Grid the auto-sized device grid with its tile count in parentheses. Sizes are read from each design's baseline run log and kept in data/benchmark-sizes.json.

Design	Suite	Modules	CLBs	Memory	Mult	Grid (tiles)
ch_intrinsics	VTR 8.0	2	13	1	0	10×10 (100)
stereovision3	VTR 8.0	1	15	0	0	7×7 (49)
diffeq2	VTR 8.0	1	22	0	5	16×16 (256)
diffeq1	VTR 8.0	1	37	0	5	16×16 (256)
mkPktMerge	VTR 8.0	7	21	15	0	28×28 (784)
raygentop	VTR 8.0	15	143	1	6	19×19 (361)
sha	VTR 8.0	1	142	0	0	17×17 (289)
boundtop	VTR 8.0	13	54	1	0	15×15 (225)
mkSMAdapter4B	VTR 8.0	8	125	5	0	20×20 (400)
or1200	VTR 8.0	16	252	2	1	27×27 (729)
stereovision0	VTR 8.0	25	707	0	0	33×33 (1089)
stereovision1	VTR 8.0	16	788	0	43	40×40 (1600)
mkDelayWorker32B	VTR 8.0	16	74	43	0	50×50 (2500)
stereovision2	VTR 8.0	24	1956	0	192	82×82 (6724)
blob_merge	VTR 8.0	2	494	0	0	28×28 (784)

for wire delay and routing area; it is the primary metric of the packer experiments because their plans named it. A design is not fast only because its single slowest path is fast, since the next slowest path becomes the CPD once the first is sped up. We therefore also look at near-critical connections, those within 10% of the CPD in the routed timing report. VPR defines the criticality of a connection from its slack, normalised by the CPD [2]; VPR 9 computes it relaxed per timing constraint [6]. Only one near-critical statistic exists in this paper (the experiment of Section V-F); worst slack per object is not yet measured (Section IV-F).

Power. Power is not measured in any experiment of this paper. We make no claim about it, and we cite no relation of power to area.

D. Hypotheses, experiments and verdicts

Each experiment subsection of Section V has the same four parts: the hypothesis in one sentence, the experiment (designs, arms, seeds), the result (numbers and the test), and the verdict. The verdict rule of each experiment is a numeric rule written in its plan file under docs/plan/; a rule is only changed by a dated entry that keeps the old text, and where a rule was changed the original verdict is the one reported. The four verdicts are:

- useful: supported under the experiment's rule, and not fragile.
- possibly good: supported, but narrow or fragile (few designs, a clause added to the rule after the data, or a rule chosen after the data). It appears in the main text only with its fragility stated and is never the sole proof of an innovation.
- inconclusive: the numbers do not decide. An experiment whose plan fixed no threshold in advance is inconclusive wherever the numbers do not decide; we never invent a threshold afterwards.
- dismissed: not supported (failed). Dismissed ideas get one row in the research log (Appendix A) and are argued here only where the paper's question needs the answer.

E. Statistics

Ratios are geometric means over designs of the variant over the reference; below 1 favours the variant for every metric of this paper. Where a rule uses a test, the sign test counts designs that improve, the Wilcoxon test is on the paired differences (on log ratios, one-sided where stated), the Holm adjustment corrects for the family of tests the plan names, and a bootstrap interval resamples the designs (10 000 resamples, seed 20261026; 90% or 95% as stated). A permutation null shuffles labels. Designs are not independent draws from a population: an interval is over this set of designs.

F. Mímir as the instrument

Mímir III stores, for each placed object, its provenance, its coordinates and its timing, so each experiment above can be asked of the data as well as run. What it holds today for this work: structure_mapping rows per design, structure, run and level, with the bounding box and spread of an HDL structure's CLBs and the structure's share of the critical path (crit_share, only with a timing run); and the run fields vpr.wirelength, vpr.chan_width, vpr.cpd, vpr.worst_slack, vpr.logic_clbs and the Norns fields (norns.purity, norns.reproduction, norns.cut, norns.retention, norns.critical_matched_delay_share). What is missing: near-critical statistics per HDL object (only the single worst path gives a share, so the near-critical experiment of Section V-F cannot be recomputed from Mímir), worst slack per structure, and power, which has no field at all.

V. Experiments

Each subsection states one hypothesis, the experiment, the result and the verdict. Numbers are from the files named in the comments of this source and in docs/paper/hypotheses.md. Tests that were not decided by the numbers are inconclusive; the full list, including every dismissed hypothesis, is in Appendix A.

A. Does the standard packer scatter the HDL's registers? (H8, H10a, H9a)

Hypothesis (H8). The standard packer scatters the words of the HDL across CLBs: median word purity at most 0.80 and median split factor at least 1.5.

Experiment. The 15 designs of Table II, stock packing, word purity and split factor against a null that assigns the same atoms at random to CLBs of the same sizes (50 samples).

Result. Median word purity 0.63 and median split factor 4.00; purity is above the random null on 15 of 15 designs.

Verdict. useful.

Hypothesis (H10a). Io-words give a word to LUTs that the front end's words left without one.

Experiment and result. On the same 15 designs, the median gain in the share of LUTs that carry a label is 24.1%, positive on 15 of 15 designs.

Verdict. useful.

Hypothesis (H9a-word, H9a-wordio). The Norns packer raises word purity with at most 5% CLB growth.

Experiment. 15 designs, three VPR seeds, source word and source word+io, β from a grid chosen on the four smallest designs (which read only purity and CLB count).

Result. Purity rises by 0.111 (word) and 0.104 (word+io), and by 0.120 and 0.124 without the four grid-choice designs; CLB count change 0.0%; Holm-adjusted Wilcoxon $p < 0.001$ for both.

Verdict. useful, as a mechanism check: purity is not an end metric. (Source module: purity gain 0.000, inconclusive.)

B. How much room is there for a better packer? (H24, H27, H29–H31, H25/H19, H32, H20)

Hypotheses. The yardstick is exact: synthesis keeps the construction and the known arm reproduces the intended blocks (H24, H27, H29, H30, H31). The standard packer is worse than the known clustering by at least 5% (H25, H19, H32). The Norns packer is still worse than the known clustering (H20).

Experiment. The yardstick designs of Section III-B: 72 designs at the netlist level, 63 of the 72 at the HDL level that the label rule keeps, 105 alignment designs, arms stock, norns and known, three VPR seeds.

Result. The known arm reproduces the blocks exactly (mean reproduction 1.000, 189 HDL runs and 216 netlist runs); the alignment manipulation works (Spearman 0.988 between designed alignment and measured purity; purity 0.100 at $a = 0$ and 1.000 at $a = 1$) and synthesis keeps the construction on 105 of 105 alignment designs. Stock over known, geometric mean: on the 72 netlist designs CLB 1.073, WL 1.220 [1.168, 1.275] (90% interval), CPD 1.093; on the 63 HDL designs CLB 1.073, WL 1.219, CPD 1.082; on the mesh and tree alignment designs (14 cells) CLB 1.152, WL 1.489 [1.394, 1.591], CPD 1.181; Holm $p < 0.001$ throughout. The room depends on the topology: the stock WL ratio is 1.42 to 1.53 on mesh and tree and close to 1.0 on chain and random graphs, so the pooled 1.22 mixes designs with room and designs without. For the Norns packer on the 63 HDL designs, WL is still 1.099 [1.057, 1.147] of the known clustering (Holm $p = 0.003$); CLB (1.031) and CPD (1.032) are inconclusive. These HDL designs include the dp designs whose word file is the leak (Section V-D), so H20 describes the packer with the hint; it does not say how much room remains with realistic words.

Verdict. useful (validity, H25/H19, H32, and H20 for WL). Limits of the yardstick are in Section VI-A.

C. How far is the packer from a known optimum on real designs? (H38, round 9)

Hypothesis (H38). On real designs made into known-optimum yardsticks (Section III-C), the standard packer is worse than the known clustering by at least 5% in CLBs, wire or delay. The rule is that of the synthetic yardstick: room if the geometric mean of stock over known is at least 1.05 and the 90% bootstrap interval of the mean log ratio excludes 0; no room if the mean is at most 1.02 or the interval lies inside [0.98, 1.02]; otherwise inconclusive. With fewer than 12 designs no pooled claim is made.

Experiment (pilot). Three designs (ch_intrinsics, diffeq1, mkPktMerge: 12 to 42 clusters, with memory slices and multipliers), each built with the own and the foreign clustering, three shuffles, three VPR seeds, arms stock and known (108 runs), plus 18 stock runs on the unshuffled ordered files. Flow flags and architecture as the synthetic yardstick (VTR commit 3c9a4d23b2 [3], [15]). Table III.

Result. The known arm needs 1.02 to 1.11 times the exact CLB count N_C and reproduces 40% to 80% of the clusters; the standard packer needs 1.06 to 1.19 times N_C. The room is small on two designs and larger on the smallest: stock over known is 1.13 in CLBs and 1.09 to 1.14 in wire on ch_intrinsics, between 1.007 and 1.075 on the other two, and the delay ratio is between 0.98 and 1.07 (room only on ch_intrinsics with the own clustering). No design shows the 1.49 of the synthetic meshes and trees. The ordered control has stock purity within 0.06 of the shuffled arm on all six builds, higher by at most 0.04 and lower once (0.80 against 0.85 on ch_intrinsics own), so file order gives the packer no visible advantage. The wire gap is larger on the foreign build than on the own build

TABLE III
H38 pilot: stock over known (geometric mean over 3 shuffles × 3 seeds) and CLB counts against the exact bound N_C. Status by the rule of the text: R room, N no room, I inconclusive. Source: data/r9/pilot-summary.json.

Design	C	N_C	stock/N_C	known/N_C	CLB	WL	CPD
ch_intrinsics	own	12	1.19	1.06	1.13 R	1.09 R	1.06 N
ch_intrinsics	foreign	12	1.19	1.06	1.13 R	1.14 R	0.98 N
diffeq1	own	42	1.14	1.11	1.02 I	1.01 N	1.02 N
diffeq1	foreign	41	1.11	1.10	1.02 N	1.03 I	1.00 N
mkPktMerge	own	22	1.06	1.03	1.03 I	1.01 N	0.99 N
mkPktMerge	foreign	22	1.09	1.02	1.07 R	1.02 N	1.02 N

for all three designs (by 0.05, 0.03 and 0.01), which is the direction a small advantage to VPR on its own clustering would take, but three designs cannot separate self-bias from design.

Verdict. inconclusive (3 designs; the rule needs 12). The row is the pilot and not the answer. The comparison with the synthetic yardstick is a pointer only: on these three designs the artificial regularity did not hide a larger gap.

Full set. Plain English. On 33 real designs the standard packer needs about 10% more CLBs than the number of clusters N_C of the known clustering, about a third more wire than the placement from that clustering, and a few per cent more delay. The own clustering (the one VPR's packer made) gives the standard packer no visible advantage over the foreign one. Two of the 35 designs (stereovision0, stereovision2) are not in the set: the generator did not finish on the second and was not started on the first.

Experiment. 13 VTR Verilog designs and 20 MCNC designs (33), each built with the own and the foreign clustering, three shuffles, three VPR seeds, arms stock and known (Norns packer told C), plus an exact arm that packs each cluster alone and assembles the clusters (25 of the 33 designs; the other eight have hard blocks, constants or chains the assembled netlist cannot hold). N_C is a counting lower bound on the CLBs of D'; where VPR packs the written file with a few small LUT atoms the generator did not count, or sweeps adders, it is corrected to the demand of the netlist VPR packed (N_C corrected, Section III-C). The design is the unit: geometric mean over designs, 90% bootstrap interval over designs (10000 resamples), Holm over CLB, WL and CPD, rule as in the hypothesis. The stock arm is the Norns binary with no affinity file, which packs as the standard packer (its .net equals the stock .net on the 45 census runs of the critical-heap study; the gate was not repeated on the round 9 designs).

Result. Stock over N_C is 1.108 [1.080, 1.149] in CLBs on the own clustering and 1.101 [1.073, 1.140] on the foreign one; with the corrected N_C it is 1.103 [1.078, 1.137] and 1.096 [1.071, 1.130], and with the 13 designs whose atom counts differ left out it is 1.10 and 1.09. Against the exact arm (25 designs) stock needs 1.099 times the CLBs, 1.322 [1.278, 1.367] times the wire and 1.033 times the delay on the own clustering (1.087, 1.330 and 1.054 on the foreign one). Against the known arm (33 designs) the ratios are 1.059, 1.265 [1.223, 1.307] and 1.026 on the own clustering (1.056, 1.287, 1.042 on the foreign one). Every CLB and WL row has Holm $p < 0.0001$ and passes the room rule; CPD is inconclusive against known (the mean lies between 1.02 and 1.05) and inconclusive (own) or room (foreign) against exact. The synthetic yardstick gave 1.073, 1.220 and 1.093 against the known arm (Section V-B), so the real structure gives about the same area gap, a larger wire gap and a smaller delay gap. Table IV.

Self-bias. The own over foreign ratio of stock is 1.007 [0.999, 1.014] in CLBs against N_C (Holm $p = 0.14$); against the exact arm it is 1.011 in CLBs (Holm $p = 0.011$), 0.994 in WL (p 0.55) and 0.980 [0.962, 0.997] in CPD (Holm $p = 0.09$, which the rule counts as excluding 1); against known the WL ratio 0.983 does not survive Holm ($p = 0.19$). Stock needs about 1% more CLBs relative to the bound on its own clustering and reaches about 2% closer to the exact delay on it; every row is within 2%, far below the gaps above.

Limits. clma has $N_C = 51$ against 100 CLBs for stock (1.95) and 96 for known: C is not a strict local optimum there and the gap is not explained; without it stock over N_C is 1.089 (own) and 1.081 (foreign). 13 designs are flagged because VPR's atom counts differ from the generator's by a few LUTs (the identity LUTs VPR absorbs, constants) or swept adders (stereovision1); a few builds add an external input pin to a cluster in the fill step, so for them the cluster graph is not kept exactly (the cut does not change). The exact arm covers the easier 25 designs. CPD is observed and not claimed optimal.

Verdict. H38 is useful (confirmed as a finding about the yardstick): the pooled stock over N_C in CLBs is 1.10 with Holm $p < 0.0001$ and an interval above 1, and the same holds for WL against known and exact. The evidence meets the test of possibly good (Holm $p < 0.05$, interval excludes 1), but the label is kept for claims about a packer and this is a measurement of room; delay is inconclusive.

D. Does the packer pay when the words equal the blocks? The leak (H33, H36, H35, H34, H22, H23)

Hypotheses. (H33) With words equal to the blocks the Norns packer beats stock. (H36) Its gain grows with the measured alignment. (H35) It makes WL worse by at least 2% when the words do not match the blocks. (H34) It beats stock at each alignment below 1. (H22) On the 63 HDL designs of the first yardstick round it beats stock.

Experiment. The 105 alignment designs; 14 designs per alignment level (mesh and tree, seven sizes); Norns over stock, three seeds.

Result. At $a = 1$ (words equal blocks) Norns/stock is CLB 0.870, WL 0.669 [0.627, 0.713], CPD 0.841 (H33).

TABLE IV

H38 full set: stock over reference (geometric mean over designs, 90% bootstrap interval over designs). R room, I inconclusive by the rule of the text. Source: data/r9/full-summary.json.

Ratio	C	n	CLB	WL	CPD
stock/N_C	own	33	1.108 [1.080, 1.149] R		
stock/N_C	foreign	33	1.101 [1.073, 1.140] R		
stock/N_C corrected	own	33	1.103 [1.078, 1.137] R		
stock/N_C corrected	foreign	33	1.096 [1.071, 1.130] R		
stock/exact	own	25	1.099 [1.065, 1.146] R	1.322 [1.278, 1.367] R	1.053 [1.032, 1.074] R
stock/exact	foreign	25	1.087 [1.055, 1.136] R	1.330 [1.280, 1.384] R	1.054 [1.037, 1.076] R
stock/known	own	33	1.059 [1.049, 1.071] R	1.265 [1.223, 1.307] R	1.026 [1.014, 1.038] I
stock/known	foreign	33	1.056 [1.046, 1.066] R	1.287 [1.243, 1.332] R	1.042 [1.027, 1.062] I
synthetic, stock/known	–	72	1.073	1.220	1.028

TABLE V

H22′: Norns over stock, mesh and tree pooled (14 designs per width), geometric mean with the 90% bootstrap interval; and the chain control (7 designs). Below 1 favours Norns. Source: data/analysis/r8-results.json.

w	WL [90%]	CPD	CLB	chain WL
4	1.652 [1.525, 1.809]	1.329	1.033	3.575
8	2.011 [1.824, 2.237]	1.371	1.012	5.604
16	2.349 [2.125, 2.609]	1.380	0.973	7.969
32	2.503 [2.242, 2.802]	1.407	0.946	9.497

Below 1 it is worse: WL 2.034, 2.038, 1.731 and 1.218 of stock at $a = 0$, 0.25, 0.5 and 0.75 (Holm $p < 0.001$ for harm at every level; H35), so no leak-free level shows a gain (H34). The slope of the WL ratio on the measured alignment is negative in 14 of 14 cells (median -1.317; -0.786 on the leak-free levels only; Holm $p < 0.001$; H36). The first yardstick round had reported WL 0.902 [0.866, 0.936] of stock on 63 HDL designs (H22). That gain sits only in the dp designs, whose word file equals the block partition (purity 0.9998), and is zero in the 27 ctl designs, where the Norns .net equals the stock .net on 27 of 27; the purity gain is zero when the HDL has no vector (median difference 0.000; H23).

Verdicts. H33, H36, H35: useful; H33 is an upper bound, because the words equal the blocks, and H35 is a finding about harm. H34: dismissed. H23: dismissed as stated. H22: dismissed: its gain is withdrawn as a result. It was an oracle leak and is reported here only as a finding about method.

E. With realistic words, does the packer reduce wirelength? (H22′, H37)

Hypothesis (H22′). With bus words that do not mirror the CLB partition, the Norns packer reduces WL against stock.

Experiment. 84 generated designs (Section III-E): mesh and tree, four word widths w, seven sizes (14 designs per width), and chain controls (seven per width); arms stock, norns ($\beta = 4$) and known; three seeds; 798 runs, all completed. Two controls ran first: at $\beta = 0$ the Norns .net must equal stock's (28 of 28 chain designs byte-identical), and at alignment 1 on chain Norns/stock must be about 1.00 (1.003 [0.993, 1.013]; both passed). The hypothesis is supported if the WL geometric mean is at most 0.99 with Holm $p < 0.05$, and not supported if it is at least 1.00 or its 90% interval lies inside [0.98, 1.02].

Result. Table V. The packer raises WL by 65% to 150% at every width, and CPD by 33% to 41%; Holm-adjusted $p = 1.0$ at every width. It reduces the CLB count by 2.7% and 5.4% at $w = 16$ and 32 (Holm $p = 0.0015$, 0.0010) and raises it at $w = 4$ and 8; a CLB result never changed the WL verdict. The chain control fails at every width, so no mesh or tree figure may be read as a gain. The known grouping reaches 0.672 of stock WL on mesh and tree at every width: the headroom is real, and realistic words do not reach it.

The chain control, per alignment level (H37). The first control, pooled over alignment levels, gave Norns/stock 4.15 on 35 chain designs, and was read as a failed control. The pooled number averages over levels. Per level (chain, WL Norns/stock, $n = 7$ each): $a = 0$ 8.71, $a = 0.25$ 8.37, $a = 0.5$ 6.19, $a = 0.75$ 2.72 and $a = 1$ 1.003. At $a = 1$ the packer leaves chain unchanged. The harm grows as word purity falls (median purity 0.10, 0.26, 0.51, 0.75, 1.00). The pooled control was ill-posed: chain has no headroom, not no effect, and a packer that follows misaligned words is expected to do harm there. The H22′ word files have median purity 0.26 at $w = 4$ down to 0.07 at $w = 32$, and the harm grows with w in the same way. The mechanism is consistent with the packer pulling flip-flops of different blocks into one CLB because they share a word; it was not measured directly (no per-CLB mixing was counted, and width, purity and words per block change together).

Verdicts. H22′: dismissed (not confirmed at any of the four widths). H37: dismissed, as a control that was ill-posed; the per-level figures above replace it. Not tried: a smaller β, words that only partly agree with the blocks, and real HDL words (Section VII).

F. Are near-critical connections kept inside CLBs? (H17c)

Measurement (H17a, not a hypothesis). Over 15 designs with stock packing, the median share of CLB pairs that are series-critical is 4.0%, shared-input or orthogonal 32.0%, and series-noncritical or filler 64.4%.

Hypothesis (H17c). Near-critical edges (within 10% of the CPD) cross CLB boundaries less often than edges in general under the standard packer.

Experiment. The 9 designs with at least 20 near-critical hops, routed timing reports.

Result. Median rate ratio 0.652 (near-critical over all edges), one-sided Wilcoxon $p = 0.027$.

Verdict. possibly good. It passes the rule as written, on 9 designs; it would fail a minimum of 12 designs that was not part of the rule text, and the design set is small. It

is the only near-critical statistic in this paper. The Norns packer does not change the series-critical share (median change −0.003; H17b, dismissed).

G. What is the critical heap, and where does it go? (CH-a to CH-d)

Plain English. While it packs, the greedy packer keeps going back to the connections that look nearly as slow as the slowest. We call that set the critical heap. The picture is that the packer works the heap, a new heap forms as it goes, and at the end a heap remains whose criticality has been pushed down to a floor it cannot get below. We ask four things: how big the heap is, which HDL constructs end up in it, whether the heap that the packer sees is the heap that exists after routing, and whether anything known from the HDL before packing predicts it. These are measurements. No decision rule was set before them, so they carry no verdict; the hypotheses they suggest are untested ones (Section VII).

Experiment. The 45 routed runs of the keep sweep (the 15 Verilog designs of Table II, the E10 base flow, VPR seeds 1 to 3, VTR commit 3c9a4d23b2 [3], [15]), each with the 1000 most critical paths kept; the criticality of an atom is the largest criticality of a reported path through it. The pack-time side is a log of the packer's own criticality on the same flags (a census whose .net is byte-identical to the sweep run in 45 of 45 runs). Constructs come from the HDL operator, block and netlist-detector views of the structure store. Summaries are the median over designs of the per-design mean over seeds, with a 90% bootstrap interval over designs. In the default flow VPR computes pack-time criticality once, before packing, with one delay for every net between logic blocks, and never updates it (the only call that refreshes it is in the analytic global placer); the first write-up of this study said otherwise and was corrected, and none of its numbers change.

Result (CH-a, size and shape). In no run does the report stop above criticality 0.90, 0.95 or 0.99, so the heap is complete. The post-route heap holds a median 4.6% of a design's atoms, lying in a median of 14 register-bit fan-in cones (11% of the combinational atoms). The number of reported paths within 0.01 of the worst ranges from 1 to 25 over the designs.

Result (CH-b, constructs and bits). With whole-design baselines, carry-chain adders are enriched only at the very top of the heap: 2.18 [1.17, 2.45] at criticality at least 0.99 ($n = 11$ designs), against 1.05 [0.51, 1.37] at 0.90. Multipliers are strongly enriched (4.60 [0.96, 8.23]) but occur in two designs. Equality comparators are absent from the heap in most designs (median enrichment 0.00), and multiplexers and relational operators lie below their share of atoms (0.36 and 0.37). Neither word bits nor carry-chain positions show a most-significant-bit effect: the heap rate by position quartile in adder chains is 0.034, 0.044, 0.031 and 0.034.

Result (CH-c, dynamics). The packer does take the pack-time heap earlier (Spearman between cluster number and pack-time criticality −0.35 [−0.43, −0.11]), but the heap does not stay put: only 0.23 [0.11, 0.45] of the pack-time heap is in the post-route heap, while 0.76 [0.31, 0.94] of the post-route heap was in the pack-time heap, so about a quarter of it arrives from below 0.90. The floor is high: with wire delay removed the slowest reported path would run at 0.40 [0.37, 0.44] of the real CPD, and wire is 0.63 [0.56, 0.65] of the delay on near-critical paths. A heap always exists, because criticality is relative to the worst path, and the wire share is what placement decides.

Result (CH-d, HDL hints). A leave-one-design-out logistic model on construct kind and expression depth predicts membership in the post-route heap at an AUC of 0.49 [0.38, 0.60] (0.5 is no signal); with netlist depth and register and adder type added, 0.57 [0.44, 0.65]. For the pack-time heap the netlist depth alone reaches 0.73 [0.63, 0.79], better than construct kind (0.70). Grouping the atoms of one register bit's fan-in cone covers a near-critical path completely for a minority of paths only (a median 11% of the near-critical paths lie inside the cone of their own endpoint).

Verdict. Measurements (CH-a to CH-d). In plain terms: the heap is small and concentrated, most of what the packer takes as critical is not in the routed heap, and the HDL constructs we can name before packing hardly predict the routed heap. Limits: only the 1000 reported paths enter (a deeper unreported path may exist); word and bit attributions exist only for atoms in a reported path; the construct and keep runs are separate Odin III runs of the same netlist, with incomplete overlap; two designs hold multipliers; the 20 MCNC designs have no attribution and are not in the study.

H. How does the greedy pick go wrong, and what are the exception HDL patterns? (HP-1 to HP-4)

Plain English. If the packer's idea of what is critical is out of date, it spends its early picks on the wrong atoms. We replay the packer's own log against the routed result and ask where its picks go wrong, and, because the HDL is what we can see before packing, we keep a catalogue of the constructs for which the pick is known to go wrong. A pattern in the catalogue is an exception HDL pattern: a detectable construct for which grouping by HDL structure is a bad packing hint. The aim is that each can be found by a detector that runs before packing and mapped to a hint for the packer. Every hint is untested. These too are measurements and carry no verdict.

Experiment. The same 45 runs as CH-a to CH-d, no new VPR run. A pick is a molecule added to a cluster; the data are the packer's queue, gains and refusals. Summaries as before ($n = 15$ designs unless stated).

Result (HP-1, stale criticality; P1, P6). Picks made on pack-time criticality of at least 0.90 are 4.3% [3.0%, 11.1%]

of all picks, and 81% [63%, 91%] of them are stale: they hold no atom of the post-route heap (74% of seed picks and 82% of candidate picks). In the other direction 32% [15%, 62%] of the post-route heap is packed in the second half of the cluster order, and 5% [1%, 7%] had a pack-time criticality below 0.50. A related effect: 10% [0%, 25%] of the picks made on a timing gain of at least 0.90 are atoms whose own input pins are all below 0.50, the gain being inherited from another sink of the net; those stay in the heap 4% [0%, 17%] of the time, against 14% [6%, 17%] for picks with a critical pin of their own.

Result (HP-2, capacity, filler and ties; P2, P4, P5). A near-critical path (criticality at least 0.90) touches a median 3.7 [3.0, 4.3] clusters against a floor from capacity of 1.9 [1.0, 2.7]; 98% [82%, 100%] of the paths are above that floor, and 98% [85%, 100%] of the distinct crossing edges fall where the earlier cluster closed with all 10 logic elements used: almost every cut is a capacity cut. What fills the cluster is mostly not the heap: in clusters that hold a post-route heap atom, 70% [64%, 74%] of the logic elements hold atoms outside it. The queue gain of the picked molecule equals that of another queued molecule in 35% [29%, 45%] of the picks (60% within 5%), and 98% of exact ties go to the molecule queued first.

Result (HP-3, forks and fan-out; P3, P7). At a fork atom (two or more successors on near-critical paths) the clusters split the successors in 67% [33%, 90%] of the cases ($n = 11$); 66% [26%, 83%] of the crossing edges sit at a fork or join atom. Nets of more than 32 sinks, which the packer's gain ignores, are almost absent from the near-critical nets (0.0% [0.0%, 0.2%]; one net in all 15 designs), but nets of 9 to 32 sinks, treated as ordinary nets, carry 24 near-critical nets in 7 designs, driven by comparison results, subtract borrow bits, state bits and enables.

Result (HP-4, HDL words; P8). Of 165 word groups of at least 8 bits (36 from the Odin III attribution, 129 from top-level buses) 91% have under 1% of their bit pairs tied by dataflow (a net, or one atom in between). The standard packer already puts only 7% of the pairs of a word in one cluster and spreads a word over 3.8 times the fewest clusters it could occupy. Grouping a word as a unit therefore asks the packer to bind bits that have no dataflow between them; the words with the most tied pairs (5% to 25%) are adder words, where the carry chain does tie the bits.

The exception HDL pattern catalogue. Table VI lists the 11 patterns. The patterns with a measured effect on the pick or the cut are rows 1, 3, 4, 5, 8 and 9; rows 6, 10 and 11 are measured non-problems or rare, and rows 2 and 7 occur in few designs. Frequencies count the 15 VTR Verilog designs only and say nothing about other design families. Every detector uses only the netlist, the Odin III record and VPR's own pre-pack timing analysis; for row 9 the detector measured here is the census outcome, and its pre-pack form is not yet counted.

Verdict. Measurements (HP-1 to HP-4). The catalogue is the output of the study: it says where the greedy pick is known to go wrong and what a hint would have to say. Limits: whether a cut was avoidable needs a packer run (the log records the candidates proposed, not the ones that could have been); the Odin III word attribution covers only atoms in a reported path (36 words of 8 atoms or more), so port buses stand in for the rest; ties are reconstructed from queue-time gains; only the 1000 reported paths enter; and designs with few near-critical paths (ch_intrinsics, mkPktMerge, stereovision1) give thin numbers.

I. Does refreshing criticality while packing lower the delay? (H-refresh)

Hypothesis (H-refresh). The stale picks of HP-1 come from a criticality that never moves. Refreshing the pack-time criticality while packing, after every closed cluster ($k = 1$) or every eighth ($k = 8$), with connections that became internal given an intra-cluster delay estimate, lowers the post-route CPD against stock without raising CLBs or wirelength by more than 2%.

Experiment. An option in the Norns packer (–norns_refresh_crit k, default 0 = off) runs VPR's own incremental timing update after every k-th closed cluster. A connection inside a closed cluster gets VPR's own chain-molecule estimate of intra-cluster routing; a cut connection keeps the inter-cluster estimate. The seed order is recomputed at each refresh; the gains of molecules already in a candidate queue are not refreshed (stock behaviour). Gate: at $k = 0$ the .net is byte-identical to the stock sweep on 35 of 35 designs and 3 seeds (105 of 105; the option absent, 35 of 35). Then 35 designs (the 15 VTR Verilog designs and 20 MCNC circuits), seeds 1 to 3, arms stock, $k = 1$ and $k = 8$, E10 base flags, kept timing: 315 runs, all routed. Per design the geometric mean over seeds of the same-seed ratio; pooled over designs with a 90% bootstrap interval; Holm adjustment over the 12 tests. The rule: useful if the CPD upper bound is below 1.000, Holm $p < 0.05$, the CPD geomean is below 1 in both suites and CLBs and wirelength are at most 1.02; dismissed if the CPD interval lies inside [0.98, 1.02] or wholly above 1; inconclusive otherwise (the full rule is in docs/plan/refresh-crit.md).

Result. Table VII. CPD does not move (1.005 [0.990, 1.021] at $k = 1$, 1.007 [0.997, 1.018] at $k = 8$). The near-critical endpoint counts fall (to 0.84 to 0.88 at criticality 0.90), but no test survives the Holm adjustment (smallest $p = 0.20$). Cost: pack time is 1.87 times stock at $k = 1$.

Mechanism check. Recomputing HP-1 for each arm: the share of heap picks that are stale is 81% [63%, 91%] for stock, 88% [69%, 91%] at $k = 1$ (up 3.6 points [0.0, 8.0]) and 79% [73%, 93%] at $k = 8$ (up 0.5 points). The refresh doubles the share of picks that are heap picks (7.7% to 15.8%; the reason is not diagnosed), but those picks are no better matched to the routed heap. So the packer now

TABLE VI
The exception HDL pattern catalogue (design counts out of 15). Detectors use only what exists before packing. Every hint is untested. Source: docs/plan/heap-picking.md, data/analysis/heappick.json.

#	Pattern (HDL shape)	Detector	Frequency	Hint for the packer
1	Parallel bus word: reg [31:0] q; q <= d; bit i depends on bit i of another word only	A bus of 8 bits or more whose logic atoms have under 1% of pairs tied by dataflow	91% of 165 words, 13 designs	Do not group the word; group by bit slice
2	Carry-chain adder word: assign s = a + b;	8 or more adder cells chained carry-in to carry-out	11 designs; median 2 chains	Group the word, start the chain at a cluster boundary; expect one cut per 20 adders
3	Fork atom with divergent critical branches: x=f(a); y=g(x); z=h(x);	Atom with 2 or more consumers on pins of pack-time criticality ≥ 0.90	13 designs; median 40 atoms	Keep the fork with the consumer of larger downstream depth; accept the cut on the other
4	Capacity cut at a cluster filled by non-heap atoms (general, not an HDL exception)	A pack-time-critical path of 2 or more BLEs	Every design: 98% of paths above the floor	Reserve slots: stop adding non-critical candidates while a critical successor is pending
5	Decision-bit fan-out: if (a > b) w <= x; a state or enable bit steering a word	Net of 9 to 32 sinks, driven by a comparison, carry-out or register, with a pre-pack-critical sink	8 designs; 24 near-critical nets in 7	Pull the sinks of a decision bit into the driver's cluster in criticality order
6	Very high fan-out control (enable, reset, select of a wide word)	Net with more than 32 sinks	17 nets per design; near-critical in 1 net	Group its sinks by bit slice; low priority
7	Deep non-adder cone (longer than a cluster)	Combinational LUT level ≥ 11 on a non-adder atom	1 design (blob_merge, level 22)	Expect $\lceil \text{depth}/10 \rceil - 1$ cuts; place them on the lowest-fan-out nets
8	Hard-block-fed logic: p = a * b; r <= p + c;	Logic atom with an input net driven by a multiplier or memory	10 designs; 7.4% of logic atoms	Discount pre-pack criticality through a hard block; do not spend heap picks on it
9	Timing gain inherited from a sibling sink	Driver of a net whose largest sink criticality is ≥ 0.90 and whose connecting sink is < 0.50	10% of picks on a high gain	Use the criticality of the connecting pin, not the maximum over the net
10	Equal-criticality sibling set: bit-parallel logic fed by one net	Net with 3 or more sinks of identical pack-time criticality ≥ 0.90	6 designs; ties on 35% of picks	Break ties by downstream depth; weak evidence
11	Collector or status register read only at a pad	Flip-flop whose output feeds no logic atom	6 designs; 0% of flip-flops (median)	None needed; use as filler

TABLE VII
H-refresh: refreshed over stock, geometric mean over 35 designs with the 90% bootstrap interval (below 1 is better). Holm p over 12 tests. Source: data/analysis/refresh-crit-stats.json.

Metric	$k = 1$	p	$k = 8$	p
CPD	1.005 [0.990, 1.021]	1.00	1.007 [0.997, 1.018]	1.00
Endpoints ≥ 0.90	0.879 [0.776, 0.995]	0.94	0.840 [0.736, 0.953]	0.20
Endpoints ≥ 0.95	0.859 [0.725, 1.009]	0.98	0.841 [0.705, 1.000]	0.94
Endpoints ≥ 0.99	0.862 [0.743, 0.994]	0.94	0.988 [0.878, 1.119]	1.00
Wirelength	1.003 [0.997, 1.009]	1.00	1.004 [0.995, 1.014]	1.00
CLBs	1.002 [0.997, 1.007]	1.00	1.005 [1.000, 1.010]	0.98
Pack time	1.865 [1.620, 2.172]	–	1.147 [1.104, 1.196]	–

sees a moving heap, and picking from it did not help; the criticality moved the wrong way for the hypothesis.

Verdicts. H-refresh at $k = 1$: inconclusive: no CPD gain, but the interval's upper bound, 1.0205, is just outside the dismissal band, so the rule does not reach dismissed. At $k = 8$: dismissed: the CPD interval lies inside [0.98, 1.02]. The hypothesis is not supported at either k. Limits: the seed refresh is part of both arms and the candidate-side refresh was not separated; the intra-cluster delay estimate is VPR's chain-molecule estimator (out to the cluster boundary and back in), which likely overestimates a pure crossbar hop, and a smaller estimate was not tried.

J. What do VPR's options do to area and speed? (E10)

Hypotheses. Nine one-factor hypotheses around the base flow (the annealing placer, the default ABC script, $K = 6$, the k6_frac architecture): two other ABC scripts; $K = 4$ (which also removes the hard blocks); the EArch architecture; unrelated clustering forced on and forced off; VPR's analytic placer; nextpnr HeAP and SA through the flat-placement file.

Experiment. 35 designs (20 MCNC, 15 from Table II), three seeds, one factor against the base. The comparisons were fixed, no thresholds were. We call a measure supported only if its 95% design-bootstrap interval excludes 1.000 and its Holm-adjusted p (36 tests) is below 0.05.

Result. Table VIII. The area and speed measures move in different directions. The area-oriented ABC script lowers the LUT count by 3.0% and raises the CPD by 14.1%. The EArch architecture lowers the CLB count by 14.5% and raises WL by 16.0%. Unrelated clustering lowers the CLB count by 7.6% with no supported change in CPD or WL. The analytic and nextpnr placers lower WL by 4.3% to 5.0% and raise the CLB count by 3.1% to 4.0%, with the CPD not decided. Forcing unrelated clustering off changes nothing: all four ratios are exactly 1.000.

Verdict. Eight of the nine are possibly good: each has at least one measure that survives the adjustment, but the

verdict rule is post hoc, the design set is not a sample of a population, $K = 4$ is confounded with the removal of the hard blocks, and the external placers inherit VPR's hard blocks and pads (rule H0) and nextpnr has no timing model. The last (forced off) is dismissed as a no-op. Measures whose interval excludes 1.000 but whose Holm p is not below 0.05 (for example the LUT count and CPD of ABC script 1) are inconclusive.

K. Does starting placement from the designer's grouping help? (R1, H4)

Hypothesis (R1a). A seeded start gives better placement than VPR's own analytic start when the anneal begins at VPR's own temperature.

Experiment. The 15 designs of Table II; seven seed families (Urd and six recovered, Section III-G) in a soft arm with the full intra-cluster order or shuffled; one VPR seed.

Result. Table IX. Soft seeds stay within a few percent of the baseline: geomean WL 0.997 to 1.029 across families, 1.009 pooled, with the channel width lower (0.947 to 0.994) and the placement runtime 1.07 to 1.09 times longer. Hard floorplan regions are worse (geomean WL 1.088 over 104 runs; R1c, dismissed). The crossover sweep finds a median crossover of 16× (range 4× to 256×), and at twice the crossover the seeds win only 57 of 105 head-to-heads against an unseeded run at the same temperature (R1e). The best placement over all seeds, P^*, is the best of about 20 placements and no control restarts the anneal with equal effort, so part of the crossover may be selection. A random start is as good as any seed: with Urd the WL ratio is 0.997 from a random start, 1.020 from permuted clusters and 1.029 from the quotient layout (H4, dismissed).

Verdict. R1a and R1e: inconclusive. The honest answer is that seeding at VPR's own temperature has no measured gain on 15 designs and one VPR seed, and this is why the paper moved to the packer.

L. Is the hierarchy physical on the chip? (H11)

Hypothesis. CLB pairs of the same word, or of the same module, sit closer on the annealed placement than their net connectivity alone predicts.

Experiment. Stock packing, analytic start, three VPR seeds; the stratified mean distance difference of related over unrelated pairs as a fraction of the unrelated distance (negative is closer), against a permutation null.

Result. Same-word median -0.176 ($p = 0.017$; 9 of 12 designs sign-consistent, 12 required); same-module median -0.188 ($p < 0.001$; 10 of 11, 12 required).

Verdict. inconclusive: the pairs are closer, but the count of designs that the rule requires cannot be reached on the 11 or 12 designs that qualify.

M. How much would hindsight gain, and do other placers agree? (R7a, R8a, R8b)

Hypothesis (R7a). Re-clustering according to where the best stock placement put atoms lowers WL×CPD on held-out seeds.

Experiment. 1660 VPR runs; the arm is chosen on seeds 1–5 and measured on seeds 6–10; 15 designs, 8 with a chosen arm.

Result. Held-out median WL gap 0.987, CPD gap 0.994, CLB gap 1.026; the objective interval lies above 1 on 1 of 8 designs (7 borderline); 7 of the 15 designs are imputed as 1.

Verdict. inconclusive: little evidence of room that a better clustering with hindsight would capture.

Hypotheses (R8a, R8b). nextpnr HeAP lands where VPR's analytic placer does, and nextpnr SA places with equal or less wirelength than VPR's annealer.

Experiment. 13 eligible designs, seeds 4–6 held out, ratios within a family against the reread control, design bootstrap.

Result. HeAP over reread: WL 1.001 [0.984, 1.019], CPD 1.027 [1.009, 1.046], WL×CPD 1.028 [0.996, 1.061]. SA over reread: WL 0.947 [0.919, 0.975], CPD 1.003 [0.973, 1.039], WL×CPD 0.952 [0.909, 1.003].

Verdict. Both inconclusive: the WL×CPD intervals include 1; rule H0 means the outside placers inherit VPR's hard blocks and pads; neither outside placer is timing-driven.

N. Is there a seed that makes the packer rebuild the known clustering? (H-seed, H-seed-b)

Hypothesis (H-seed-b). For most known clusters there is a seed atom from which stock VPR's growth rebuilds the cluster exactly, and seeding with it closes most of the area gap to the known optimum N_C.

Experiment. The packer is given the known clustering of a round-9 design. At each new cluster it grows a trial cluster from every atom of the next known cluster with VPR's own gain and legality, undoes the trial, and commits the seed whose trial equals the known cluster (else the best Jaccard). 33 designs, 3 VPR seeds. As T-VPack [16] we report speed and density separately, speed first.

Result. 25.5 percent of the known clusters are rebuilt exactly (mean Jaccard 0.61). Stock over oracle, geometric mean over designs with the 90 percent interval: critical-path delay 0.993 [0.983, 1.004] (oracle better on 13 of 33 designs), wirelength 1.028 [1.017, 1.040] (25 of 33), CLBs 1.012 [1.006, 1.019] (23 of 33). The oracle closes 11 percent of the log gap to N_C (1.118 to 1.105).

Verdict. Peter's hypothesis, that the seed oracle does better than the base (H-seed-b), is possibly good in wirelength (a 2.8% geomean win, 25 wins and 8 losses of 33) and in area (CLBs 1.2%, 23 wins, 6 losses, 4 ties),

TABLE VIII

E10: ratio of each level to the base flow (annealing placer), geometric mean over designs; * marks a measure whose 95% interval excludes 1.000 and whose Holm $p < 0.05$ (supported); = marks a byte-identical ratio. Below 1 is better. $n = 35$ except $K = 4$ (33–34) and the nextpnr arms (30 on CLBs, CPD and WL). Source: data/e10/ratios.csv.

Level	LUTs	CLBs	CPD	WL
ABC script 1 (rewriting)	1.103	1.114*	1.027	1.113*
ABC script 2 (area-oriented)	0.970*	0.970	1.141*	0.985
$K = 4$ architecture	1.974*	5.320*	1.488*	1.288
EArch architecture	1.000=	0.855*	1.004	1.160*
unrelated clustering on	1.000=	0.924*	0.996	1.001
unrelated clustering off	1.000=	1.000=	1.000=	1.000=
VPR analytic placer	1.000=	1.040*	1.005	0.957*
nextpnr HeAP	1.000=	1.032*	1.011	0.950*
nextpnr SA	1.000=	1.031*	1.018	0.955*

TABLE IX

Seeding baseline: geomean quality ratios against the unseeded baseline, soft arm (15 designs, one VPR seed); the inter-cluster-only row shuffles the order inside each cluster and pools all seven families, and the k-means row pools k, $2k$ and $4k$. Below 1.0 is better. Source: docs/plan/EXPERIMENT.md (round one).

Seed	Wirelength	Delay	Chan. width	Runtime
Urd (hierarchy)	1.029	1.004	0.958	1.068
Louvain	1.008	0.982	0.980	1.091
Spectral	1.006	0.993	0.968	1.098
Affinity propagation	1.002	1.003	0.982	1.086
k-means (k, $2k$, $4k$)	1.006	0.985	0.975	1.088
Inter-cluster only	1.015	0.981	0.987	1.102

and inconclusive in speed (the critical-path delay is 0.7% worse, 13 wins and 20 losses): a trade-off. The strong form (H-seed-b2), that most known clusters can be rebuilt from some seed and that this closes most of the gap, is dismissed: 25.5% of the clusters are rebuilt exactly and 11% of the log gap to N_C is closed, so the gap is in growth (gain and closing), not seeding. The earlier rule-based seed choice (H-seed) is a 0.7% geomean CLB win, inconclusive.

O. What is the packer bad at? (Q-N2)

Question. Where, and for which kind of decision, do the clusters of VPR's default packer (APPack) depart from the known clustering C? A measurement, no hypothesis.

Experiment. The packer's decision log (molecule added, queue contents with gain, refusals, close) of 264 stock packs (33 round-9 designs, own and foreign C, 4 shuffles) is replayed against C. Each half-empty cluster hands its lost slots to the first deviation on its own timeline; leftover pieces of split known clusters are handed back to the kind of pick that took the other members away. No VPR run.

Result. The stock/N_C ratio is 1.07 at the median design (10th to 90th percentile 1.03 to 1.17). 60 percent of the CLB gap (66 percent without clma) comes from picks of an atom of another known cluster while a legal member of the seed's own known cluster was queued, with a strictly lower queue gain in 99.8 percent of them (mean ratio 0.46; ties 0.14 percent). 11 percent comes from clusters closed with free slots after input-pin refusals (24 percent on the VTR Verilog designs, 0.3 percent on MCNC); a capacity cut never occurs. Of the near-critical edges that cross stock clusters but not known clusters, 59 percent trace to the same wrong adds; the nets cut are 1.33 times those of C (median design).

Verdict. measurement. Distance attenuation of APPack is not measured (no global-placement positions were logged). How to make it better is the next question.

P. Is there data in the circuit that ranks the right atom? (Q-N3)

Question. At the picks of Q-N2 where the packer took an atom of another known cluster although a legal member of the seed's own known cluster was queued, is there a number, computable from the circuit and the cluster built so far (never from C), that ranks the right atom above the pick? A positive answer would be a candidate group-aware gain term. A measurement, no hypothesis.

Experiment. The 26,214 wrong picks per shuffle of Q-N2 are split into connectivity picks (37%) and timing-pull picks (63%; the pick's timing gain beats the right atom's). Eight features (shared nets, cluster atoms within two hops, common neighbours, absorbed nets, new input pins, fan-out, sequential-level gap, shared near-critical edges) and a combined score fitted on 17 tune designs are scored on 16 held-out designs by the pairwise AUC and the top-1 of the right atom, against VPR's own gain; a second view deletes the dummy cells of the construction. No VPR run.

Result. On the connectivity picks the cluster atoms within two hops, the shared nets and the combined score reach AUC 0.69, 0.68 and 0.69 (90 percent intervals 0.66 to 0.71, 0.64 to 0.71, 0.66 to 0.72; VPR's gain 0.003 by construction), and over all decisions the combined score picks a member of the seed's known cluster 6.8 points more often than VPR's gain. With the dummy cells deleted the features fall to 0.53 to 0.61, the combined score to 0.59, and it no longer beats VPR on the connectivity decisions (-5.2 points). The winning seeds of the Q-N1b oracle are not the most critical, central or best-connected members

of their clusters; in 85 percent the first candidate in file order already rebuilds the cluster.

Verdict. measurement. No feature passes the rule fixed before the analysis; a weak two-hop connectivity signal remains that this benchmark cannot separate from its construction (the fill nets cannot be removed). Candidate gain terms are untested ideas.

Q. Does a two-hop connectivity term make APPack better on real designs? (H-2hop)

Hypothesis. The Q-N3 feature that ranked the right atom best (the number of cluster atoms within two hops of the candidate), added to APPack's candidate gain, lowers the post-route critical-path delay on real designs without raising CLBs or wirelength by more than 2 percent.

Experiment. Gain $= (1 - w)\times$ stock $+ w\, h_2$, with h_2 the cluster atoms within two hops of the molecule divided by the cluster's atoms, applied before APPack's distance attenuation; $w = 0.1$ and 0.25, fixed before the runs. The 35 designs of E10 (15 Verilog, 20 MCNC), VPR seeds 1 to 3, 315 runs; with $w = 0$ the packed netlist is byte-identical to stock on all 105 packs.

Result. Speed first. Critical-path delay, geomean win over stock with the 90 percent interval: $w = 0.1$ +0.7 percent $[-0.6, +1.9]$ (17 wins, 14 losses, 4 ties), $w = 0.25$ −1.9 percent $[-4.0, 0.0]$ (15, 17, 3). The count of endpoints with criticality of at least 0.90 falls 14 and 19 percent, but no test survives the Holm correction (smallest $p = 0.29$). Wirelength −0.2 and −1.0 percent (13 to 15 wins of 35) and CLBs −0.6 and +0.1 percent are within 1 percent. Pack time rises 5 to 6 percent.

Verdict. $w = 0.1$ is inconclusive: a 0.7 percent geomean delay win (17 wins, 14 losses, 4 ties), not separable from noise at this sample size; wirelength 0.2 percent and CLBs 0.6 percent worse (a trade-off). $w = 0.25$ is dismissed: its delay interval lies above 1.000 (worse). The term that ranks the right atom better offline does not help as a gain on real designs.

R. Does loosening the clustering inside the stock square grid buy speed? (H-spare)

Hypothesis. Peter's reading of Schabas and Brown [17]: after stock clustering fixes the (square) FPGA, the spare CLB tiles can be spent leaving s BLEs empty in the most timing-critical clusters, and this lowers the post-route critical-path delay inside the same grid without raising wirelength by more than 2 percent. This is the packing change only; duplication after placement is not built.

Experiment. Pass 1 reads each design's stock grid (square on all 35) and its spare CLB tiles; the grid is then fixed for every arm. A logic cluster whose seed has pre-pack criticality at least T (at most $n = \lfloor \text{spare} \times 10/s \rfloor$ such clusters, capped at the stock clusters) closes at $10 - s$ BLEs (an occupied fle); the rest is stock. Arms stock, $s = 2$, $s = 4$; the 35 designs of E10 (the 20 MCNC are the 20 circuits of Schabas's Table 1), VPR seeds 1 to 3; two channel-width settings (the E10 flow at each arm's own minimum width, and every arm at 1.2× the stock minimum width, Schabas's setting): 630 runs. With $s = 0$ the packed netlist is byte-identical to stock on 105 of 105 packs.

Result. Speed first. Post-route delay, geomean win over stock with the 90 percent interval, E10 flow: $s = 2$ −1.4 percent $[-3.1, +0.2]$ (12 wins, 21 losses, 2 ties), $s = 4$ −1.5 percent $[-2.8, -0.2]$ (10, 23, 2); at fixed width −1.6 percent $[-2.7, -0.6]$ and −2.1 percent $[-3.4, -0.9]$. The 20 MCNC alone: −1.4 and −1.4 percent (E10 flow), against Schabas's +6.4 percent for the same packing change. The placed (pre-route) delay is worse as well (−2.1 and −2.8 percent), as in their Table 2, but routing recovers only 0.7 to 1.3 points of it, not enough to change the sign; the minimum width falls 0.8 and 1.7 percent. Wirelength is 2 to 3 percent worse and CLBs used 11 to 17 percent higher (the grid is fixed); the near-critical endpoint counts do not move reliably. The qualified clusters hold no more of the near-critical atoms than their share of the design (55 against 57 percent at $s = 2$, 41 against 37 percent at $s = 4$). Designs with a large spare (grid set by IOs or hard blocks) fare no better. One design (seq) does not fit at $s = 2$ and counts as a loss; VPR's own re-pack (unrelated clustering) fires in 43 percent of the loosened runs.

Verdict. $s = 4$ is dismissed in both settings (delay interval wholly above 1.000, worse). $s = 2$ is inconclusive in the E10 flow (a 1.4 percent geomean loss whose interval reaches 1; wirelength 2.2 and CLBs 10.8 percent worse) and dismissed at fixed width. Schabas's packing gain is not reproduced on a modern flow (K6 fracturable LUTs, timing-driven APPack, annealing placer).

VI. Discussion and Limits

What the experiments say. The designer's grouping is physically real on the chip (H11, inconclusive by the letter of the rule), and the standard packer loses it (H8). A packer that is told the grouping can keep it (H9a), and when the grouping is exactly the right clustering it uses far less wire (H33: 0.669 of stock). But the same packer, handed registers that cut across the blocks, makes wirelength worse by 65% to 150% (H22′), and the harm falls as the registers line up with the blocks (H37 per level, H35, H36). The known-optimum yardstick shows the room is real: the known grouping reaches 0.672 of stock wirelength on meshes and trees at every register width (H22′) and the standard packer is 1.49 times it (H32). The honest reading is that in these experiments the gain appeared only when the hint was perfect: no alignment below 1 showed one, and our packer at $\beta = 4$ does not degrade gracefully.

The leak as a method lesson. The original gain (0.902 of stock) was real arithmetic on a flawed input: the word file was the answer. The yardstick caught it because it lets us dial the alignment, and because we reran with words that cut across blocks. A packer or placer claim on constructed benchmarks should always be accompanied by a control in which the hint is useless and one in which it is perfect.

Multi-metric view. Area and speed move in different directions: the area-oriented ABC script lowers the LUT count and raises the CPD (E10); the outside placers lower WL and raise the CLB count; the packer lowers CLBs at wide words and raises WL and CPD (H22′). A result on one metric is not a result on the problem. Speed is judged here by the CPD and, only once, by the near-critical statistic (H17c). Power is not measured.

The critical heap. The packer does not see a moving heap: its criticality is computed once before packing, with one delay for every net, so the heap it works on is the pre-pack heap, and 81% of its picks on that heap hold nothing that is critical after routing (HP-1). Refreshing the criticality while packing made the packer see a new heap, and its picks were no better matched to the routed one (H-refresh). Most cuts of near-critical paths are capacity cuts at full clusters (HP-2), and the HDL constructs we can name before packing hardly predict the routed heap (CH-d, AUC 0.49). The catalogue of exception HDL patterns (Table VI) is the list of what a hint would have to handle, and every hint in it is untested.

Yardsticks. The synthetic yardstick has almost no Rent growth, so its gap (1.22 in wire over all topologies) is a gap on regular designs. The first read of the real-design yardstick, on three designs, shows smaller gaps (between 1.007 and 1.135 in wire), not larger: the artificial regularity did not visibly hide a bigger gap on these three. Three designs decide nothing, and the full set is pending.

A. Limits

- Few, small designs. The real designs are 15 small VTR circuits; the seeding baseline used one VPR seed per configuration. Intervals are over this set of designs and not over a population.
- Constructed designs. The yardstick designs and the H22′ words are generated. The word layout of H22′ is a random permutation, the extreme of misalignment; real buses may lie between that and the leak, and are untested (Section VII).
- The yardstick is a clustering optimum. It is exact for CLB count and not a proven placement optimum. Synthesis keeps the construction in only 32 of 72 HDL designs under the original label rule (63 under an amended rule), and the netlist-level zero-violation check passes on 37 of 72 (H18, H28, dismissed by their literal rules); the analysis uses the 63 designs the amended rule keeps, and the netlist-level variants chosen after the data are labelled.
- An erratum. The affinity files of the first packer experiments left out every adder (carry-chain) atom; H9 and H10 describe a packer that did not see carry chains. The later experiments (H22′ and the alignment designs) use corrected files.
- Near-critical evidence is thin. H17c has 9 designs and possibly good; Mímir cannot yet recompute it (Section IV-F).
- Outside placers. Rule H0 and the lack of a timing model in nextpnr bias every comparison against VPR's own placers.
- E10. The verdict rule is post hoc, $K = 4$ is confounded with the removal of the hard blocks, and some arms exclude designs where a run failed.
- The real-design yardstick is a pilot on three designs with a single-swap margin for the cut and a checked, not proved, global optimum; up to 14 cells per build could not be anchored, and for diffeq1 the independent partitioner check is vacuous. The heap studies cover the 15 VTR Verilog designs and the 1000 most critical paths of each run.
- Power is not measured.

VII. Future Work: Untested Hypotheses

Ten ideas have no result yet. Each is stated with the one experiment it needs.

U1: a smaller β. A smaller affinity weight may keep words without the harm seen at $\beta = 4$ on misaligned words. Experiment: re-run H22′ on the same 84 designs over a ladder of β below 4 with the same rule.

U2: partly aligned words. Words that agree only partly with the blocks, may give a gain or a smaller harm. Experiment: re-run H22′ with words drawn at a controlled partial alignment on the realistic bus layout.

U3: real HDL words. Real HDL words may behave like the constructions. Experiment: run stock and norns on the 15 VTR designs with the word files as written and test WL with the H22′ rule.

U4: crossover versus selection. The crossover of the seeding baseline may reflect the seed and not selection of the best of about 20 placements. Experiment: an equal-effort control that restarts the anneal from the best of N unseeded runs.

U5: packing for near-critical paths. A packer that weights near-critical connections may improve the near-critical statistics. Experiment: per-object near-critical statistics in Mímir (the top-k or within-10% paths) and a packer arm that uses them.

U6: reserve room on the path. Holding logic elements open for a pending critical successor may lower the clusters a near-critical path touches (HP-2: 98% of cuts are capacity cuts at full clusters). Experiment: a packer arm that stops adding non-critical candidates while such a successor is pending; measure clusters per path against the capacity floor, and the CPD.

U7: bit-slice instead of word-major grouping. Nine of ten words have no dataflow between their bits (HP-4), so grouping a parallel bus by bit slice may beat grouping it as a unit, while carry-chain words stay grouped. Experiment: word-major against bit-slice affinity per word class on the 15 VTR designs.

U8: the connecting pin's criticality. Using the criticality of the pin that connects to the cluster, not the maximum over the net's sinks, may cut stale timing-gain picks (catalogue row 9). Experiment: a packer arm with pin-specific criticality; measure the stay rate of those picks.

U9: the other catalogue hints. Keeping the fork atom with the deeper branch, pulling decision-bit sinks to the driver, discounting paths through hard blocks and breaking ties by downstream depth each have a detector in Table VI. Experiment: one packer arm per hint on the designs where its detector fires.

U10: bit-path affinity. Giving the packer the fan-in cone of each register bit as the affinity file may lower the CPD and the near-critical endpoint counts. Experiment: cone groups at several β, cones of the deepest 5%, and a random-group null on the 15 designs, three seeds.

VIII. Conclusion

Keeping the designer's grouping through packing is a sound idea and a measurable one: the standard packer scatters the source's registers, a packer with an affinity bonus keeps them, and a yardstick with a known best grouping shows real room. But we could not show a gain from it. The one gain we measured came from a leak, and with realistic registers the packer used more wire at every setting we ran. Starting placement from the designer's grouping gave no measured benefit either. What the work does deliver is the instrument: the yardstick with an adjustable hint, the controls that expose a leak, and the run records that Mímir stores so that area, speed and, once it is measured, power can be traced back to the HDL that caused them.

Appendix A
Research Log

Every hypothesis of the work, one row each, including the dismissed ones. The full register, with the experiment, metrics, test and source file of each row, is docs/paper/hypotheses.md. The counts are 15 useful, 9 possibly good, 22 inconclusive, 18 dismissed and 9 measurements (H17a, CH-a to CH-d, HP-1 to HP-4), 73 rows in all, 64 of them hypotheses with a verdict. A row that groups related hypotheses (for example H34a–d) counts once; the measurements CH-a to CH-d and HP-1 to HP-4 are separate rows of the register and are listed here in two grouped rows.

References

[1] P. A. Jamieson, "Improving the Area Efficiency of Heterogeneous FPGAs with Shadow Clusters," Ph.D. dissertation, University of Toronto, 2007.

[2] J. Luu, "Architecture-Aware Packing and CAD Infrastructure for Field-Programmable Gate Arrays," Ph.D. dissertation, University of Toronto, 2014.

[3] J. Rose, J. Luu, C. Yu, O. Densmore, J. Goeders, A. Somerville, K. Kent, P. Jamieson, and J. Anderson, "The vtr project: architecture and cad for fpgas from verilog to routing," in Proceedings of the ACM/SIGDA international symposium on Field Programmable Gate Arrays, 2012, pp. 77–86. [Online]. Available: http://dl.acm.org/citation.cfm?id=2145708

[4] J. Luu, J. Goeders, M. Wainberg, A. Somerville, T. Yu, K. Nasartschuk, M. Nasr, S. Wang, T. Liu, N. Ahmed, K. B. Kent, J. Anderson, J. Rose, and V. Betz, "VTR 7.0: Next generation architecture and CAD system for FPGAs," ACM Trans. Reconfigurable Technology and Systems, vol. 7, no. 2, pp. 6:1–6:30, Jun. 2014.

[5] K. E. Murray, O. Petelin, S. Zhong et al., "VTR 8: High-performance CAD and customizable FPGA architecture modelling," ACM Trans. Reconfigurable Technology and Systems, vol. 13, no. 2, 2020, article 9.

[6] M. Wainberg and V. Betz, "Robust Optimization of Multiple Timing Constraints," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 34, no. 12, pp. 1942–1953, 2015.

[7] P. Jamieson, "Loki III: Reverse engineering structural netlists without a language model," 2026, manuscript.

[8] M. Gort and J. H. Anderson, "Analytical placement for heterogeneous FPGAs," in Proc. Int. Conf. Field Programmable Logic and Applications (FPL), 2012, pp. 143–150.

[9] D. Shah, E. Hung, C. Wolf, S. Bazanski, D. Gisselquist, and M. Milanović, "Yosys+nextpnr: An open source framework from Verilog to bitstream for commercial FPGAs," in Proc. IEEE Int. Symp. Field-Programmable Custom Computing Machines (FCCM), 2019, pp. 1–4.

[10] F. Gharibian, L. Shannon, and P. Jamieson, "Finding system-level information and analyzing its correlation to FPGA placement," in Proc. Int. Conf. Field Programmable Logic and Applications (FPL), 2010, pp. 544–549.

[11] F. Gharibian, L. Shannon, P. Jamieson, and K. Chung, "Analyzing system-level information's correlation to FPGA placement," ACM Trans. Reconfigurable Technology and Systems, vol. 6, no. 3, pp. 15:1–15:21, 2013.

[12] F. Gharibian, L. Shannon, and P. Jamieson, "Identifying and placing heterogeneously-sized cluster groupings based on fpga placement data," in Field Programmable Logic and Applications , 2014 24th International Conference on. IEEE, 2014, pp. 1–6.

[13] J. Cong, M. Romesis, and M. Xie, "Optimality and stability study of timing-driven placement algorithms," in Proceedings of the 2003 IEEE/ACM international conference on Computer-aided design, ser. ICCAD '03, 2003, pp. 472–. [Online]. Available: http://dx.doi.org/10.1109/ICCAD.2003.108

[14] M. A. Elgammal, K. E. Murray, and V. Betz, "RLPlace: Using reinforcement learning and smart perturbations to optimize FPGA placement," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 41, no. 8, pp. 2532–2545, Aug. 2022.

[15] M. A. Elgammal, A. Mohaghegh, S. G. Shahrouz, F. Mahmoudi, F. Koşar, K. Talaei, J. Fife, D. Khadivi, K. Murray, A. Boutros, K. B. Kent, J. Goeders, and V. Betz, "VTR 9: Open-source CAD for fabric and beyond FPGA architecture exploration," ACM Transactions on Reconfigurable Technology and Systems, vol. 18, no. 3, pp. 1–53, 2025.

[16] A. Marquardt, V. Betz, and J. Rose, "Using Cluster-Based Logic Blocks and Timing-Driven Packing to Improve FPGA Speed and Density," in ACM/SIGDA International Symposium on FPGAs, Monterey, CA, 1999, pp. 37–46. [Online]. Available: http://doi.acm.org/10.1145/296399.296426

[17] K. Schabas and S. D. Brown, "Using logic duplication to improve performance in FPGAs," in Proc. ACM/SIGDA Int. Symp. Field Programmable Gate Arrays (FPGA), 2003, pp. 136–142.

TABLE X
Research log (1 of 3): every hypothesis, one row each. Verdicts as defined in Section IV-D. Numbers are from the files named in docs/paper/hypotheses.md.

ID	Hypothesis	Result	Verdict
Rounds 1 and 2 (seeding)			
R1a	Seeded start beats VPR's start at its own temperature	WL 0.997 to 1.029 across 7 families; pooled 1.009; one seed	inconclusive
R1b	Provenance (Urd) and recovered clusters seed differently well	Urd soft 1.029 (worst), Urd hard 1.062 (best hard); order flips	inconclusive
R1c	Hard floorplan regions help	Hard WL 1.088 over 104 runs, worse than soft	dismissed
R1d	Intra-cluster order matters beyond the coarse layout	WL 1.009 (full) vs 1.015 (inter), mixed signs	inconclusive
R1e	Cluster-derived start keeps winning when the anneal starts hotter	Median crossover 16×; seeds win 57 of 105 at 2×	inconclusive
R1f	Checkpointing extends the useful temperature range	Same crossover on 14 of 15 designs	dismissed
H7a	Word seeds beat module seeds	WL 0.986, better on 10 of 15; sign test $p = 0.302$	inconclusive
H7b	Bit-sliced layout beats plain word clusters	WL 0.982 ($p = 0.791$); channel width 1.033	inconclusive
H7c	Hierarchy plus bit-slices has the lowest WL of 11 families	Ranks 8 of 11 (1.014)	dismissed
H7d	Seeding only the word CLBs beats the full word seed	WL 0.980, better on 9 of 15; $p = 0.607$	inconclusive
H4	Order random > permuted > quotient ≥ analytic start	Urd 0.997, 1.020, 1.029: a random start is as good as any seed	dismissed
H1	VPR's placement is module- and word-coherent	Above null+2sd on 8/11 (module) and 9/15 (word) designs; needs 2/3	inconclusive
H2	Seed gain correlates with design structure	0 of 9 correlations positive at $p < 0.05$	inconclusive
Round 3 (packing dilutes structure; the Norns packer)			
H8	Stock packing scatters HDL words	Word purity 0.63, split factor 4.00	useful
H9a-module	Affinity packing raises module purity	Purity +0.000, CLB −1.6%	inconclusive
H9a-word	Affinity packing raises word purity, CLB growth ≤ 5%	Purity +0.111, CLB 0.0%, $p < 0.001$ (mechanism check)	useful
H9a-wordio	Same with word+io source	Purity +0.104, CLB 0.0% (mechanism check)	useful
H9b	Affinity packing alone changes WL	WL 1.051 / 1.032 / 1.030 (module / word / word+io), p 0.09 to 0.19	inconclusive
H9c	Affinity packing plus word seed beats stock plus word seed	WL 1.064 / 1.039 / 1.039, better on 2 to 5 of 15	dismissed
H10a	Io-words label LUTs that had no word	Median coverage gain 24.1%, 15 of 15 designs	useful
H10b	Word+io affinity beats word-only	WL 0.998, better on 4 of 15	dismissed
H11	Same-word and same-module pairs sit closer than connectivity predicts	Word −0.176 ($p = 0.017$), module −0.188 ($p < 0.001$); design counts below the rule	inconclusive

TABLE XI
Research log (2 of 3): every hypothesis, one row each. Verdicts as defined in Section IV-D. Numbers are from the files named in docs/paper/hypotheses.md.

ID	Hypothesis	Result	Verdict
Round 4 (census, hierarchy and chip size, packer with CPD)			
H17a	Pair-class split of what the packer clusters (a measurement)	Series-critical 4.0%, shared-input and orthogonal 32.0%, rest 64.4%	measurement
H17b	The Norns packer changes the series-critical share	Median change −0.003	dismissed
H17c	Near-critical edges cross CLBs less often than edges in general	Median ratio 0.652, $p = 0.027$, 9 designs; fails an n<12 clause outside the rule	possibly good
H17d	Stock packer beats chance in series pairs	Median excess 0.072, 15 of 15 positive (sanity check)	useful
H13a	Module count correlates with die area	Supported under both packings	useful
H13b	Hierarchy relates to die area beyond design size	Partial rho below thresholds	inconclusive
H13c	Packer's area effect depends on module count	Not decided	inconclusive
H14	Norns packer beats stock on WL, channel width, CPD	WL 1.033, channel width 1.015, CPD 0.996; Holm $p = 1.0$	dismissed
H15	RTL depth estimate predicts critical statements	Median rho 0.137	inconclusive
H16	Stage-by-bit floorplan seed beats the analytic start	WL 1.005 / 1.015 under both packings	dismissed
K1–K8	Eight relations known from the data hold in ≥ 12 of 15 designs	K4, K5 hold; K1 10/11, K2 9/12 below the literal count	inconclusive
Rounds 5 and 6 (yardstick, leak), H22′ re-run			
H18	Synthesis keeps the construction ($\geq$ 90% of designs)	32 of 72 (63 of 72 under the amended rule)	dismissed
H28	Netlist-level construction passes the zero-violation check	37 of 72 (72 of 72 under a slack rule chosen after the data)	dismissed
H24, H27	Forced clustering reproduces the blocks	Mean reproduction 1.000, 100% exact (189 and 216 runs)	useful
H25/H19	Stock packer is worse than the known clustering by $\geq 5\%$	Netlist WL 1.220 [1.168, 1.275]; HDL WL 1.219	useful
H20	The Norns packer is still worse than the known clustering	WL 1.099 [1.057, 1.147]; CLB 1.031 and CPD 1.032 inconclusive	useful
H21/H26	The gap grows with design size	Stock WL +0.009 per doubling ($p = 0.051$); other cells not supported	inconclusive
H22	The HDL-aware packer beats stock (first yardstick round)	WL 0.902; withdrawn: an oracle leak (word file equals the blocks)	dismissed
H23	Gain only where the HDL exposes words	Gain sits in dp; ctl .net equal to stock on 27 of 27	dismissed
H29–H31	Alignment designs are valid; manipulation works; known arm exact	105 of 105 kept; Spearman 0.988; reproduction 1.000	useful
H32	Stock is worse than known on mesh and tree	WL 1.489 [1.394, 1.591], CLB 1.152, CPD 1.181	useful
H33	Norns beats stock when words equal blocks	WL 0.669 [0.627, 0.713] (the leak: an upper bound)	useful
H34a–d	Norns beats stock at alignment 0 to 0.75	WL 2.034, 2.038, 1.731, 1.218 (all worse)	dismissed
H35	Norns raises WL by $\geq 2\%$ when words do not match	Supported at $a \leq 0.75$; Holm $p < 0.001$	useful
H36	Gain grows with measured alignment	Slope −1.317, 14 of 14 cells; −0.786 leak-free	useful
H37	Control: Norns equals stock on chain	Pooled 4.150; per level 8.71, 8.37, 6.19, 2.72, 1.003: the control was ill-posed	dismissed
H22′	With realistic bus words the packer reduces WL	WL 1.652, 2.011, 2.349, 2.505 at $w = 4, 8, 16, 32$; not confirmed at any width	dismissed

TABLE XII
Research log (3 of 3): every hypothesis, one row each. Verdicts as defined in Section IV-D. Numbers are from the files named in docs/paper/hypotheses.md.

ID	Hypothesis	Result	Verdict
Rounds 7 and 8, and the option sweep E10			
R7a	Hindsight re-clustering lowers WL×CPD on held-out seeds	Held-out WL gap 0.987, CPD 0.994; interval above 1 on 1 of 8 designs	inconclusive
R7b	HDL features predict hindsight co-location beyond connectivity	AUC +0.0018 (connected), +0.0042 (uniform); ceiling-limited	inconclusive
R8a	nextpnr HeAP lands where VPR's analytic placer does	WL 1.001 [0.984, 1.019], CPD 1.027 [1.009, 1.046]	inconclusive
R8b	nextpnr SA places with equal or less WL than VPR SA	WL 0.947 [0.919, 0.975]; WL×CPD 0.952 [0.909, 1.003]	inconclusive
E10-abc1	ABC script 1 changes area and speed	CLBs 1.114 and WL 1.113 worse; LUTs and CPD not adjusted-significant	possibly good
E10-abc2	Area-oriented mapping lowers LUTs and CLBs, raises CPD	LUTs 0.970, CPD 1.141; CLBs and WL not significant	possibly good
E10-k4	$K = 4$ needs more LUTs and CLBs and is slower	LUTs 1.974, CLBs 5.320, CPD 1.488 (confounded with hard blocks)	possibly good
E10-earch	EArch changes CLBs, WL and CPD	CLBs 0.855, WL 1.160; CPD not significant	possibly good
E10-uon	Unrelated clustering on changes CLBs and quality	CLBs 0.924; CPD and WL not significant	possibly good
E10-uoff	Unrelated clustering off changes nothing	All ratios 1.000 (byte-identical)	dismissed
E10-vpr_ap	VPR analytic placer differs from the annealer	WL 0.957, CLBs 1.040; CPD not significant	possibly good
E10-heap	nextpnr HeAP differs from the annealer	WL 0.950, CLBs 1.032; CPD not significant	possibly good
E10-sa	nextpnr SA differs from the annealer	WL 0.955, CLBs 1.031; CPD not significant	possibly good
Round 9 pilot, heap studies and H-refresh			
H38	On real designs made into known-optimum yardsticks stock is worse than known by $\geq 5\%$	Full set, 33 designs: stock/N_C CLB 1.108 [1.080, 1.149] own, 1.101 foreign; stock/known WL 1.265, CLB 1.059, CPD 1.026 (inconclusive); pilot 3 designs earlier	useful
CH-a to CH-d	Size, constructs, dynamics and HDL predictors of the critical heap (measurements)	Heap 4.6% of atoms; adders enriched only at ≥ 0.99 (2.18); 0.23 of the pack-time heap stays; AUC 0.49 from construct kind	measurement
HP-1 to HP-4	Where the greedy pick goes wrong; exception HDL patterns (measurements)	81% of heap picks stale; 98% of cuts at full clusters; 91% of words have no dataflow between bits; 11-row catalogue	measurement
H-refresh $k = 1$	Refreshing criticality after each closed cluster lowers CPD	CPD 1.005 [0.990, 1.021]; stale share 88% against 81%	inconclusive
H-refresh $k = 8$	Same, every eighth cluster	CPD 1.007 [0.997, 1.018]	dismissed